

媒体与认知课程上机作业最终报告

陈天乐 无 25

2025 年 6 月 1 日

目录

1	项目背景	2
2	项目目标	2
3	实验任务	2
3.1	Task1: 图像编码器	2
3.2	Task2: 文本编码器	3
3.3	Task3: 对比损失	6
3.4	训练模型	7
3.5	Task4: 结果分析与可视化	10
3.6	改进方案	13
3.6.1	自动学习率调节	13
3.6.2	数据增强与正则化	14
3.6.3	ResNet34+BERT	15
3.6.4	ViT+Transformer	21
3.6.5	ViT+BERT	24
4	实验总结	26
5	源代码	27

1 项目背景

多模态学习（Multimodal Learning）是当前人工智能领域的研究热点，其中视觉与语言的融合任务，如图文匹配、图文检索、图文生成等，具有广泛的应用前景。

CLIP（Contrastive Language-Image Pretraining）是 OpenAI 提出的多模态预训练方法，利用图像-文本对比学习，使模型能自动将图像和文本嵌入到同一语义空间中，进而支持多种下游任务。

本项目旨在指导学生构建一个简化版本的 CLIP 模型，掌握对比学习的基本方法和图文模态对齐的核心原理。

2 项目目标

1. 搭建一个具备图文对比学习能力的基础模型；
2. 使用 PyTorch 实现图像编码器与文本编码器的双塔结构；
3. 使用 InfoNCE 损失进行图文嵌入对齐训练；
4. 分析模型在多模态语义对齐方面的效果及可视化结果。

3 实验任务

本次实验使用的是 Flickr8k 数据集，数据格式为图像 + 文字描述（1-5 条）。模型结构如下

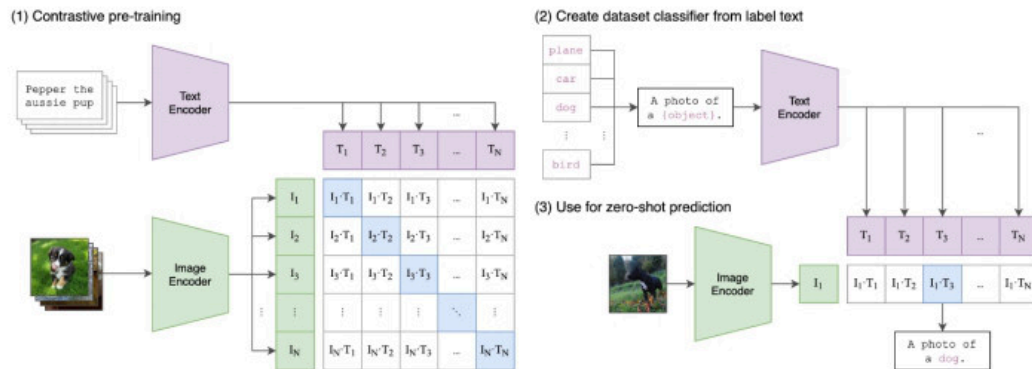


图 1: 模型结构

从示意图可以看出，该模型分为文本编码处理和图像编码处理两大部分。通过 ResNet 对图像进行特征提取，同时用 LSTM 或者 Transformer 模型（可在 `text_encoder.py` 中调整，本次实验使用 Transformer 模型）对输入文本提取文本表示，并形成图文嵌入矩阵。接着使用 InfoNCE 损失函数对图文嵌入对进行正负对比学习，

3.1 Task1: 图像编码器

该任务是将 ResNet 模型中的 forward 函数完善，代码如下：

```

1 # ----- 任务一：实现 BasicBlock 模块 -----
2
3 class BasicBlock(nn.Module):
4     def __init__(self, in_channels, out_channels, stride=1, downsample=None):
5         super().__init__()
6         # 实现第一个 3x3 卷积层, 包含 stride、padding=1, bias=False;
7         self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=3, stride=stride,
8                                 padding=1, bias=False)
9
10        # 【填空区域】：对 conv1 的输出接 BatchNorm
11        self.bn1 = nn.BatchNorm2d(out_channels)
12
13        # 定义 ReLU 激活, inplace 设为 True
14        self.relu = nn.ReLU(inplace=True)
15
16        # 【填空区域】：实现第二个 3x3 卷积层, 步长默认为1
17        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3, stride=1, padding=1,
18                                bias=False)
19
20        # conv2 的输出接 BatchNorm
21        self.bn2 = nn.BatchNorm2d(out_channels)
22
23        self.downsample = downsample
24        # 请完成forward函数
25        def forward(self, x):
26
27            identity = x
28
29            out = self.conv1(x)
30            out = self.bn1(out)
31            out = self.relu(out)
32
33            out = self.conv2(out)
34            out = self.bn2(out)
35
36            # 如果需要下采样, 对 identity 也进行转换
37            if self.downsample is not None:
38                identity = self.downsample(x)
39
40            out = out + identity # 残差连接
41            out = self.relu(out)
42
43            return out

```

3.2 Task2: 文本编码器

该任务是将 LSTM 和 Transformer 模型中的代码完善，代码如下：

```

1 # ----- 任务一：实现 BasicBlock 模块 -----
2
3 class BasicBlock(nn.Module):
4     def __init__(self, in_channels, out_channels, stride=1, downsample=None):
5         super().__init__()
6         # 实现第一个 3x3 卷积层, 包含 stride、padding=1, bias=False;
7         self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=3, stride=stride,
8                                 padding=1, bias=False)
9
10        # 【填空区域】：对 conv1 的输出接 BatchNorm
11        self.bn1 = nn.BatchNorm2d(out_channels)

```

```

12     # 定义 ReLU 激活, inplace 设为 True
13     self.relu = nn.ReLU(inplace=True)
14
15     # 【填空区域】: 实现第二个 3x3 卷积层, 步长默认为1
16     self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3, stride=1, padding=1,
17                             bias=False)
18
19     # conv2 的输出接 BatchNorm
20     self.bn2 = nn.BatchNorm2d(out_channels)
21
22     self.downsample = downsample
23     # 请完成 forward 函数
24     def forward(self, x):
25
26         identity = x
27
28         out = self.conv1(x)
29         out = self.bn1(out)
30         out = self.relu(out)
31
32         out = self.conv2(out)
33         out = self.bn2(out)
34
35         # 如果需要下采样, 对 identity 也进行转换
36         if self.downsample is not None:
37             identity = self.downsample(x)
38
39         out = out + identity # 残差连接
40         out = self.relu(out)
41
42         return out

```

```

1 class LSTMTextEncoder(nn.Module):
2     def __init__(self, vocab_size, embed_dim=256, hidden_dim=256, num_layers=1, padding_idx=0)
3         :
4         super().__init__()
5
6         self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=padding_idx)
7         self.hidden_dim = hidden_dim
8
9         self.W_i = nn.Linear(embed_dim, hidden_dim)
10        # 初始化输入门参数, U_i: 上一个隐状态至隐状态
11        self.U_i = nn.Linear(hidden_dim, hidden_dim)
12
13        # 初始化遗忘门参数, W_f: 输入至隐状态
14        self.W_f = nn.Linear(embed_dim, hidden_dim)
15        # 初始化遗忘门参数, U_f: 上一个隐状态至隐状态
16        self.U_f = nn.Linear(hidden_dim, hidden_dim)
17
18        # 初始化输出门参数, W_o: 输入至隐状态
19        self.W_o = nn.Linear(embed_dim, hidden_dim)
20        # 初始化输出门参数, U_o: 上一个隐状态至隐状态
21        self.U_o = nn.Linear(hidden_dim, hidden_dim)
22
23        # 初始化候选状态参数, W_c: 输入至隐状态
24        self.W_c = nn.Linear(embed_dim, hidden_dim)
25        # 初始化候选状态参数, U_c: 上一个隐状态至隐状态
26        self.U_c = nn.Linear(hidden_dim, hidden_dim)
27
28        # 全连接层将最终的隐藏状态映射到 embed_dim (目标嵌入空间)
29        self.fc = nn.Linear(hidden_dim, embed_dim)

```

```

30 #请完成forward函数
31 def forward(self, captions):
32     """
33     captions: [B, T], 表示批次 B 中每个句子的 token id 序列, T 为序列长度。
34     """
35     word_embedding = self.embedding(captions) # [B, T, E]
36     B, T, E = word_embedding.shape
37
38     h = torch.zeros(B, self.hidden_dim, device = word_embedding.device)
39     c = torch.zeros(B, self.hidden_dim, device = word_embedding.device)
40
41     for t in range(T):
42         x_t = word_embedding[:, t, :]
43
44         i_t = torch.sigmoid(self.W_i(x_t) + self.U_i(h)) # 输入门
45         f_t = torch.sigmoid(self.W_f(x_t) + self.U_f(h)) # 遗忘门
46         o_t = torch.sigmoid(self.W_o(x_t) + self.U_o(h)) # 输出门
47         c_hat_t = torch.tanh(self.W_c(x_t) + self.U_c(h))
48
49         c = f_t * c + i_t * c_hat_t # 更新细胞状态
50         h = o_t * torch.tanh(c) # 更新隐藏状态
51
52
53     # 将最终隐状态 h 通过全连接层映射至目标嵌入空间
54     out = self.fc(h) # [B, embed_dim]
55     # 对输出结果进行 L2 正则化归一化, 确保嵌入向量单位化
56     return F.normalize(out, p=2, dim=1)

```

```

1
2 # ----- 填空题 1: 实现 MultiHeadSelfAttention 模块 -----
3 class MultiHeadSelfAttention(nn.Module):
4     def __init__(self, embed_dim, num_heads):
5         super().__init__()
6         assert embed_dim % num_heads == 0, "embed_dim 必须能整除 num_heads"
7         self.num_heads = num_heads
8         self.head_dim = embed_dim // num_heads
9         self.qkv_proj = nn.Linear(embed_dim, embed_dim * 3)
10        self.out_proj = nn.Linear(embed_dim, embed_dim)
11
12    #请完成forward函数
13    def forward(self, x, mask=None):
14        # x: [B, T, D], D=embed_dim
15        B, T, D = x.size()
16        qkv = self.qkv_proj(x) # [B, T, 3D]
17        q, k, v = torch.chunk(qkv, 3, dim=-1)
18
19        # 拆分
20        q = q.view(B, T, self.num_heads, self.head_dim).transpose(1, 2) # [B, nh, T, hd]
21        k = k.view(B, T, self.num_heads, self.head_dim).transpose(1, 2)
22        v = v.view(B, T, self.num_heads, self.head_dim).transpose(1, 2)
23
24        #计算注意力分数
25        attn = (q @ k.transpose(-2, -1)) / math.sqrt(self.head_dim) # [B, nh, T, T]
26
27        if mask is not None:
28            attn = attn.masked_fill(mask.unsqueeze(1).unsqueeze(1) == 0, float('-inf'))
29            attn = F.softmax(attn, dim=-1)
30
31        # 计算注意力输出
32        attn_output = attn @ v # [B, nh, T, hd]
33        attn_output = attn_output.transpose(1, 2).contiguous().view(B, T, D) # [B, T, D]
34
35        #通过输出投影层输出结果

```

```

35         return self.out_proj(attn_output)
36
37 class TransformerBlock(nn.Module):
38     def __init__(self, embed_dim, num_heads):
39         super().__init__()
40         self.attn = MultiHeadSelfAttention(embed_dim, num_heads)
41         self.ln1 = nn.LayerNorm(embed_dim)
42         # 初始化前馈网络, 先 Linear(embed_dim, embed_dim*4) -> ReLU -> Linear(embed_dim*4,
43         # embed_dim)
44         self.ff = nn.Sequential(
45             nn.Linear(embed_dim, embed_dim * 4),
46             nn.ReLU(),
47             nn.Linear(embed_dim * 4, embed_dim)
48         )
49         self.ln2 = nn.LayerNorm(embed_dim)
50
51     def forward(self, x, mask=None):
52         # 残差连接: 先 LayerNorm, 再 MultiHeadSelfAttention, 再加上输入
53         # 计算自注意力块输出, 并加上原输入, 实现残差连接
54         x = x + self.attn(self.ln1(x), mask)
55         # 同理, 对于前馈网络: 先 LayerNorm, 再前馈计算, 再加上输入
56         # 计算前馈网络输出, 并加上残差
57         x = x + self.ff(self.ln2(x))
58         return x
59
60 # ----- 填空题 2: 实现 TransformerTextEncoder 模块 -----
61 class TransformerTextEncoder(nn.Module):
62     def __init__(self, vocab_size, embed_dim=256, num_heads=4, num_layers=4, padding_idx=0,
63         max_len=100):
64         super().__init__()
65         # 初始化嵌入层
66         self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=padding_idx)
67         # 初始化位置编码器, 参数为 embed_dim 和 max_len
68         self.pos_encoder = PositionalEncoding(embed_dim, max_len=max_len)
69         self.layers = nn.ModuleList([TransformerBlock(embed_dim, num_heads) for _ in range(
70             num_layers)])
71         self.fc = nn.Linear(embed_dim, embed_dim)
72
73     # 请完成 forward 函数
74     def forward(self, captions):
75         # captions: [B, T]
76
77         # 输入嵌入和位置编码
78         x = self.embedding(captions) # [B, T, D]
79         x = self.pos_encoder(x) # [B, T, D]
80
81         # 通过多层 Transformer 块
82         for layer in self.layers:
83             x = layer(x) # [B, T, D]
84
85         # 全局平均池化
86         x = x.mean(dim=1) # [B, D]
87         out = self.fc(x)
88         return F.normalize(out, p=2, dim=1)

```

3.3 Task3: 对比损失

该任务是完善 InfoNCE 损失函数, 目标是: 同一图文对相似度最大, 且不同对之间的相似度最小。代码如下:

```

1 # 请在这里写出对比损失函数
2 def contrastive_loss(image_embeds, text_embeds, temperature=0.07):

```

```

3     """
4     image_embeds, text_embeds: [batch, embed_dim]
5     """
6     device = image_embeds.device
7     batch_size = image_embeds.shape[0]
8
9     logits = torch.matmul(image_embeds, text_embeds.T) / temperature
10
11     # 创建正样本标签 [0,1,2,...,batch-1]
12     labels = torch.arange(batch_size, device=image_embeds.device)
13
14     # 计算图像->文本和文本->图像两个方向的交叉熵损失
15     loss_i2t = F.cross_entropy(logits, labels)
16     loss_t2i = F.cross_entropy(logits.T, labels) # 转置矩阵计算反向
17
18     # 最终损失取平均
19     loss = (loss_i2t + loss_t2i) / 2
20
21     return loss

```

同时，为了计算验证集中的 loss，同样编写了 loss 函数：

```

1 def calc_val_loss(img_encoder, txt_encoder, dataloader, device):
2     img_encoder.eval() # 设置为评估模式
3     txt_encoder.eval() # 设置为评估模式
4     total_val_loss = 0 # 使用 total_val_loss 累加
5     with torch.no_grad(): # 在评估阶段不计算梯度
6         pbar = tqdm(dataloader, desc="计算验证损失", unit="batch", leave=False)
7         for images, captions_ids in pbar:
8             images = images.to(device)
9             captions_ids = captions_ids.to(device)
10
11             image_embeds = img_encoder(images)
12             text_embeds = txt_encoder(captions_ids)
13             loss = contrastive_loss(image_embeds, text_embeds) # 调用对比损失函数
14             total_val_loss += loss.item()
15             pbar.set_postfix(loss=loss.item()) # 显示当前batch的损失
16     avg_val_loss = total_val_loss / len(dataloader)
17     return avg_val_loss

```

3.4 训练模型

训练流程大致为：

1. 将图像与文本数据输入模型；
2. 利用图像和文本特征提取编码器提取特征信息；
3. 归一化后计算余弦相似度矩阵；
4. 计算损失并反向传播；
5. 每轮评估 Top-1 Accuracy / Recall@K 检索准确率，并将训练中得到的最优模型保存至本地（后缀为.pth 文件）。

在 train.py 中，为了保存一个 loss 的训练曲线，需要对每个训练周期得到的 train 和 val 的 loss 保存下来，保存在数组中后，待最后一个 epoch 训练完后绘制出 loss 曲线。在每一次训练结束时输出当前训练周期的 loss、recall 等参数；每一轮训练时会检测是否已经有模型数据，这样可以从上一次的训练中断处继续训练；设置一个 improve_metric 变量，检测每一轮模型指标是否提升，如果提升则将更新的参数保存在本地。代码如下：

```

1  best_val_loss = float('inf')
2  epochs = 40
3  best_val_recall = 0
4  val_loss = 0
5  train_loss_history = []
6  val_loss_history = []
7
8  # 定义检查点路径和恢复训练标志
9  checkpoint_path = "C:/Users/skyhappy/Desktop/学校的破事/作业/媒认/媒体与认知上机作业/
    best_metric_model.pth"
10 resume_training = False # 是否从检查点恢复训练
11
12 # 初始化训练相关变量（会被检查点覆盖）
13 start_epoch = 0
14 best_val_recall = 0
15 best_val_loss = float('inf')
16 train_loss_history = []
17 val_loss_history = []
18
19 # 加载检查点
20 if os.path.exists(checkpoint_path):
21     print(f"发现检查点文件 {checkpoint_path}, 正在加载...")
22     checkpoint = torch.load(checkpoint_path, map_location=device, weights_only= False)
23
24     # 加载模型和优化器状态
25     img_encoder.load_state_dict(checkpoint['img_encoder_state_dict'])
26     txt_encoder.load_state_dict(checkpoint['txt_encoder_state_dict'])
27     optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
28
29     # 加载训练状态
30     start_epoch = checkpoint['epoch']
31     best_val_recall = checkpoint['best_val_recall_at_1']
32     best_val_loss = checkpoint['best_val_loss']
33     train_loss_history = checkpoint.get('train_loss_history', [])
34     val_loss_history = checkpoint.get('val_loss_history', [])
35
36     # 加载tokenizer词表（如果需要）
37     tokenizer.word2idx = checkpoint['tokenizer_vocab']
38
39     resume_training = True
40     print(f"成功加载检查点, 将从 epoch {start_epoch} 继续训练")
41     print(f"当前最佳 Recall@1: {best_val_recall*100:.2f}% | 最佳验证损失: {best_val_loss:.4f}")
42
43 for epoch in range(epochs):
44     img_encoder.train() # 设置为训练模式
45     txt_encoder.train() # 设置为训练模式
46     epoch_loss = 0.0
47     pbar = tqdm(train_dataloader, desc=f"Epoch {epoch+1}/{epochs}", unit = "batch")
48     for images, captions_ids in pbar:
49         images = images.to(device)
50         captions_ids = captions_ids.to(device)
51
52         image_embeds = img_encoder(images) # [batch, embed_dim]
53         text_embeds = txt_encoder(captions_ids) # [batch, embed_dim]
54         loss = contrastive_loss(image_embeds, text_embeds)
55
56         optimizer.zero_grad()
57         loss.backward()
58         optimizer.step()
59

```

```

60         epoch_loss += loss.item()
61         pbar.set_postfix(loss = loss.item())
62
63         avg_train_loss = epoch_loss / len(train_dataloader)
64         train_loss_history.append(avg_train_loss) # 记录训练损失
65         val_loss = calc_val_loss(img_encoder,txt_encoder,val_dataloader,device)
66         val_loss_history.append(val_loss) # 记录验证损失
67
68         r_txt2img, r_img2txt = evaluate_top_k(img_encoder, txt_encoder, val_dataloader, device
        , topk=(1, 5, 10))
69
70         print(f" Epoch [{epoch+1}/{epochs}]: Train Loss: {avg_train_loss:.4f} | Val Loss: {
        val_loss:.4f}")
71         print(f" 验证集 文本->图像 Recall: @1 {r_txt2img[0]*100:.2f}% | @5 {r_txt2img
        [1]*100:.2f}% | @10 {r_txt2img[2]*100:.2f}%")
72         print(f" 验证集 图像->文本 Recall: @1 {r_img2txt[0]*100:.2f}% | @5 {r_img2txt
        [1]*100:.2f}% | @10 {r_img2txt[2]*100:.2f}%")
73
74         # 如果验证集有改善, 则保存最佳模型, 这里选择loss以及recall参数
75         improved_metric = False
76         if r_txt2img[0] >= best_val_recall:
77             best_val_recall = r_txt2img[0]
78             print(f" > 验证集 Recall@1 提升至: {best_val_recall*100:.2f}%")
79             improved_metric = True
80
81         if val_loss <= best_val_loss:
82             best_val_loss = val_loss
83             print(f" > 验证集损失降低至: {best_val_loss:.4f}")
84             improved_metric = True
85
86         if improved_metric:
87             checkpoint = {
88                 'epoch': epoch + 1,
89                 'img_encoder_state_dict': img_encoder.state_dict(),
90                 'txt_encoder_state_dict': txt_encoder.state_dict(),
91                 'optimizer_state_dict': optimizer.state_dict(),
92                 'tokenizer_vocab': tokenizer.word2idx, # 保存词表
93                 'best_val_recall_at_1': best_val_recall,
94                 'best_val_loss': best_val_loss,
95                 'embed_dim': embed_dim, # 保存模型参数
96                 'vocab_size': vocab_size # 保存模型参数
97             }
98             torch.save(checkpoint, checkpoint_path)
99             print(f" > Best model updated at epoch {epoch+1} ")
100
101         print("\n训练完成。")
102
103         #绘制loss曲线
104         plt.figure(figsize=(10, 6))
105         plt.plot(range(1, epochs + 1), train_loss_history, label='平均训练损失 (Avg Train Loss)')
106         plt.plot(range(1, epochs + 1), val_loss_history, label='平均验证损失 (Avg Val Loss)')
107         plt.xlabel('轮次 (Epochs)')
108         plt.ylabel('损失 (Loss)')
109         plt.title('训练和验证损失随轮次变化曲线')
110         plt.legend() # 显示图例
111         plt.grid(True) # 显示网格
112         plt.savefig("C:/Users/skyhappy/Desktop/学校的破事/作业/媒认/媒体与认知上机作业/loss_curve.
        png") # 将图像保存到文件
113         print("损失曲线已保存为 loss_curve.png")
114         plt.show()

```

3.5 Task4: 结果分析与可视化

模型经过训练后,需要在测试集上检测其性能。设置 epoch=40, 每一个 epoch 中的 batch size 设置为 4, 最终得到的模型性能指标如下:

```
> 验证集 Recall@5 提升至: 11.30%
> Best model updated at epoch 39
Epoch 40/40: 100%
Extracting embeddings: 100%
1811/1812 [00:25<00:00, 1.70batch/s, loss=0.134] | 1812/1812 [00:45<00:00, 22.02it/s]

Test - Image Retrieval:
Recall@1: 18.81%
Recall@5: 28.73%
Recall@10: 40.30%
Epoch [40/40]: Train loss: 0.0815 | Val loss: 0.2801
验证集 文本->图像 Recall: @1 18.81% | @5 28.73% | @10 40.30%
验证集 图像->文本 Recall: @1 11.25% | @5 29.65% | @10 40.27%

训练完成
损失曲线已保存为 loss_curve.png
```

图 2: 训练 40 个周期后模型的性能指标

而绘制出的 loss 曲线如下图:

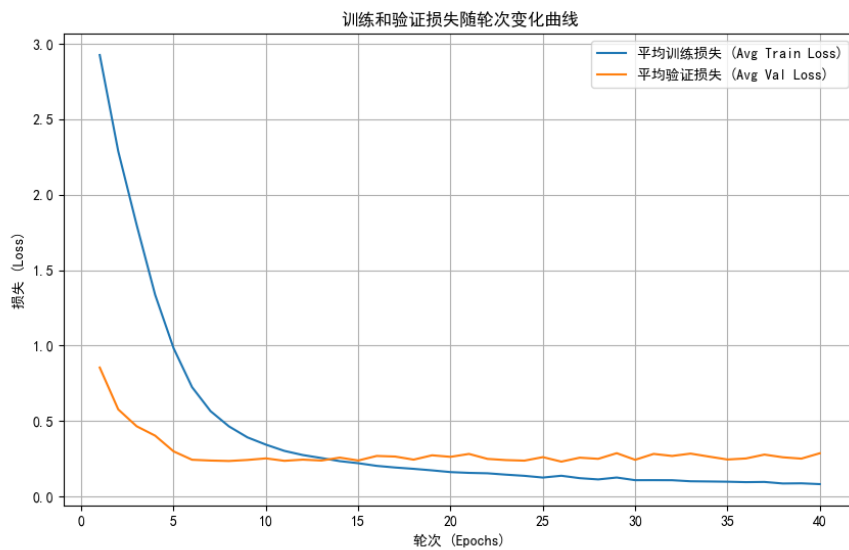


图 3: 训练 40 个周期的 loss 曲线

为了可视化文本 → 图像 Top-K 检索示例, 我新建了一个 test.py 文件, 在原本 evaluate_top_k 函数的基础上编写了 visualize_top_k 函数, 其主要实现逻辑为: 设置一个随机数, 将该随机数对应的 test_caption.txt 文件中提供的描述提取出来作为 Query, 并将相似度排名前五的图片展示出来, 代码如下:

```
1 def visualize_top_k(query_idx, img_encoder, txt_encoder, dataset, device, k=5):
2     img_encoder.eval() # 设置为评估模式
3     txt_encoder.eval() # 设置为评估模式
4
5     # 获取查询样本
6     query_image, query_caption_ids = dataset[query_idx]
7     query_caption = dataset.pairs[query_idx][1]
8
9     # 提取所有图像嵌入
10    all_image_embeds, all_img_paths = [], []
11    with torch.no_grad():
12        for img_path in dataset.img_paths:
13            image = Image.open(os.path.join(dataset.root_dir, img_path)).convert("RGB")
14            image = dataset.transform(image).unsqueeze(0).to(device)
15            embed = img_encoder(image).cpu()
16            all_image_embeds.append(embed)
```

```

17     all_img_paths.append(img_path)
18     all_image_embeds = torch.cat(all_image_embeds)
19
20     # 提取查询文本嵌入
21     with torch.no_grad():
22         caption_ids = query_caption_ids.unsqueeze(0).to(device)
23         text_embed = txt_encoder(caption_ids).cpu()
24
25     # 计算相似度
26     similarities = F.cosine_similarity(text_embed, all_image_embeds, dim=1)
27     topk_indices = similarities.argsort(descending=True)[:k]
28
29     # 可视化
30     plt.figure(figsize=(15, 5))
31     plt.subplot(1, k + 1, 1)
32     plt.imshow(Image.open(os.path.join(dataset.root_dir, dataset.img_paths[query_idx])))
33     plt.title("Query Image\n" + query_caption)
34     plt.axis('off')
35
36     for i, idx in enumerate(topk_indices):
37         plt.subplot(1, k+1, i+2)
38         img = Image.open(os.path.join(dataset.root_dir, all_img_paths[idx]))
39         plt.imshow(img)
40         plt.title(f"Rank {i+1}\nSimilarity: {similarities[idx]:.2f}")
41         plt.axis('off')
42     plt.tight_layout()
43     plt.savefig("C:/Users/skyhappy/Desktop/学校的破事/作业/媒认/媒体与认知上机作业/
44     retrieval_example.png")
45     plt.show()

```

得到的结果如下图所示：



图 4: Top-5 相似图像结果

在测试集上的性能表现如下：

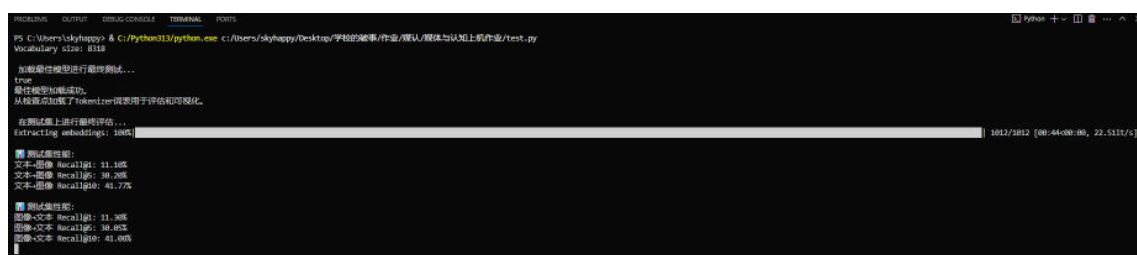


图 5: 在测试集上的模型性能

可以看到，提出的 Query 是一个和成年人一块走的小男孩，而排名前五的图片中均有小男孩出现，特别是第一和第三张图片，是大致符合 Query 要求的。剩余的三张图片只有一个小男孩，说明在检索时 query 的“little boy”肯定得到了极高的注意力得分，这可能与选择的模型以及训练过程中文本描述的多样性比较有限有关。同时，训练数据可能存在偏差，如一些场景或文本描述占比失衡，导致模型对少见情况泛化能力差。

为了提高模型表现性能，可以尝试更先进的模型结构或改进现有结构，如改进特征提取器，采用更复杂的注意力机制加强文本 - 图像特征交互；或者利用大规模公开数据集进行预训练，再在 Flickr8k 数据集上微调，提升模型泛化能力和对特定任务的适应性；还可以选用其他更为合适的对比学习损失函数，比如 Triplet loss 等等；最后还可以通过随即搜索算法、贝叶斯优化等方式对训练时超参数的设计进行优化，提高模型训练效果。

为了实现 T-SNE 嵌入空间的可视化，我在 test.py 中编写了 tsne_visualize 函数，代码如下：

```

1 def tsne_visualize(img_encoder, txt_encoder, dataloader, device, n_samples=200):
2     """TSNE嵌入空间可视化"""
3     img_encoder.eval() # 设置为评估模式
4     txt_encoder.eval() # 设置为评估模式
5
6     # 收集嵌入
7     image_embeds, text_embeds, labels = [], [], []
8     with torch.no_grad():
9         for idx, (images, captions_ids) in enumerate(dataloader):
10             if idx * dataloader.batch_size >= n_samples:
11                 break
12             images = images.to(device)
13             captions_ids = captions_ids.to(device)
14
15             image_embeds.append(img_encoder(images).cpu())
16             text_embeds.append(txt_encoder(captions_ids).cpu())
17             labels.extend([f"Sample {idx*dataloader.batch_size+i}" for i in range(len(images))
18                             ])
19
20     # 合并并降维
21     combined_embeds = torch.cat(image_embeds + text_embeds).numpy()
22     tsne = TSNE(n_components=2, random_state=42)
23     reduced = tsne.fit_transform(combined_embeds)
24
25     # 绘图
26     plt.figure(figsize=(12, 8))
27     split_point = len(image_embeds)
28     plt.scatter(reduced[:split_point, 0], reduced[:split_point, 1], c='r', label='Images')
29     plt.scatter(reduced[split_point:, 0], reduced[split_point:, 1], c='b', label='Texts')
30     plt.title("T-SNE Visualization of Embedding Space")
31     plt.xlabel("TSNE-1")
32     plt.ylabel("TSNE-2")
33     plt.legend()
34     plt.grid(alpha=0.3)
35     plt.savefig("C:/Users/skyhappy/Desktop/学校的破事/作业/媒认/媒体与认知上机作业/
        tsne_visualization.png")
36     plt.show()

```

运行后结果如图所示：

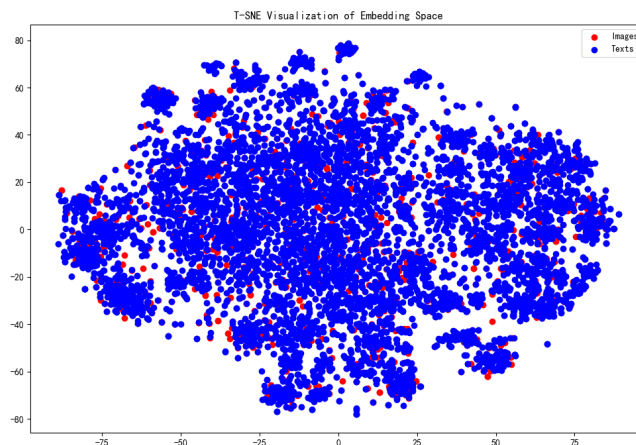


图 6: T-SNE 可视化结果

3.6 改进方案

本次实验从模型结构优化、数据增强与正则化和自动学习率调整三个方面对原有模型性能进行改进。

3.6.1 自动学习率调节

原有模型采用的是固定学习率调整策略，因此在改进方案中，我引入了“线性预热 + 余弦退火”的动态学习率调整策略，同时加入了自动早停机制防止模型过拟合，实现代码如下：

```
1 def adjust_learning_rate(optimizer, epoch, warmup_epochs = 5, base_lr = 1e-3):
2     """线性预热 + 余弦退火"""
3     if (epoch < warmup_epochs):
4         # 线性预热
5         lr = base_lr * (epoch + 1) / warmup_epochs
6     else:
7         # 余弦退火
8         progress = (epoch - warmup_epochs) / (100 - warmup_epochs)
9         lr = 0.5 * base_lr * (1 + math.cos(math.pi * progress))
10
11     for param_group in optimizer.param_groups:
12         param_group['lr'] = lr
13
14     ...
15
16     best_val_loss = float('inf')
17     epochs = 40
18     best_val_recall = 0
19     val_loss = 0
20     train_loss_history = []
21     val_loss_history = []
22     early_stop_patience = 5
23     early_stop_counter = 0
24
25     ...
26
27     avg_train_loss = epoch_loss / len(train_dataloader)
28     train_loss_history.append(avg_train_loss) # 记录训练损失
29     val_loss = calc_val_loss(img_encoder, txt_encoder, val_dataloader, device)
30     val_loss_history.append(val_loss) # 记录验证损失
```

```

31
32
33 # 查看当前学习率
34 current_lr = optimizer.param_groups[0]['lr']
35 print(f"Epoch {epoch}: Learning Rate = {current_lr:.6f}")
36
37 # 手动更新学习率
38 adjust_learning_rate(optimizer, epoch)
39
40 ...
41
42 if improved_metric:
43     checkpoint = {
44         'epoch': epoch + 1,
45         'img_encoder_state_dict': img_encoder.state_dict(),
46         'txt_encoder_state_dict': txt_encoder.state_dict(),
47         'optimizer_state_dict': optimizer.state_dict(),
48         'tokenizer_vocab': tokenizer.word2idx, # 保存词表
49         'best_val_recall_at_1': best_val_recall,
50         'best_val_loss': best_val_loss,
51         'embed_dim': embed_dim, # 保存模型参数
52         'vocab_size': vocab_size # 保存模型参数
53     }
54     torch.save(checkpoint, checkpoint_path)
55     print(f"        > Best model updated at epoch {epoch+1} ")
56     early_stop_counter = 0 # 指标改善, 重置计数器
57 else:
58     early_stop_counter += 1 # 指标未改善, 计数器+1
59     print(f"    早停计数器: {early_stop_counter}/{early_stop_patience}")
60
61 # 触发早停条件
62 if early_stop_counter >= early_stop_patience:
63     print(f"    连续 {early_stop_patience} 轮无指标改善, 提前终止训练!")
64     break # 退出训练循环
65
66 ...

```

3.6.2 数据增强与正则化

对模型训练数据进行增强, 对图像进行旋转和缩放, 提升模型鲁棒性; 同时对文本数据进行同义词替换、随机删除等方式, 增强文本描述的多样性。实现代码如下:

```

1 ...
2
3 class TextAugmenter:
4     def __init__(self, aug_src='wordnet', replace_ratio=0.2, delete_ratio=0.1):
5         # 同义词替换增强器 (基于 WordNet)
6         self.synonym_aug = naw.SynonymAug(aug_src=aug_src, aug_p=replace_ratio)
7         # 随机删除单词增强器
8         self.delete_aug = naw.RandomWordAug(aug_type='delete', aug_p=delete_ratio)
9
10    def augment(self, text):
11        # 随机选择一种增强方式 (或叠加使用)
12        augmented = self.synonym_aug.augment(text) # 同义词替换
13        augmented = self.delete_aug.augment(augmented) # 随机删除
14        return augmented

```

```

1 ...
2
3 # 训练集增强Transform (包含数据增强)

```

```

4   train_transform = T.Compose([
5       T.RandomRotation(degrees=15), # 随机旋转 (-15°~+15°)
6       T.RandomResizedCrop(size=(224, 224), scale=(0.8, 1.2)), # 随机缩放裁剪
7       T.ColorJitter(brightness=0.1, contrast=0.1, saturation=0.1, hue=0.1), # 色彩抖动
8       T.RandomHorizontalFlip(p=0.5), # 随机水平翻转 (50%概率)
9       T.ToTensor(), # 转为Tensor (0-255→0-1)
10      T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]) # 归一化 (预训练模
      型常用)
11  ])
12
13  # 验证集/测试集基础Transform (仅预处理)
14  val_transform = T.Compose([
15      T.Resize((224, 224)), # 统一尺寸
16      T.ToTensor(),
17      T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]) # 与训练集保持一致
      的归一化
18  ])
19
20  # 构建数据集与 DataLoader: 训练集、验证集、测试集
21  train_dataset = Flickr8kDataset(
22      root_dir="C:/Users/skyhappy/Desktop/学校的破事/作业/媒认/媒体与认知上机作业/Flickr8k/
      images", # 图片所在目录
23      captions_file=train_token_file, # 训练集 captions 文件, 格式: image<TAB>caption
24      tokenizer=tokenizer,
25      transform=train_transform,
26      is_train=False
27  )
28  val_dataset = Flickr8kDataset(
29      root_dir="C:/Users/skyhappy/Desktop/学校的破事/作业/媒认/媒体与认知上机作业/Flickr8k/
      images",
30      captions_file=val_token_file, # 验证集 captions 文件
31      tokenizer=tokenizer,
32      transform=val_transform,
33      is_train=False
34  )
35
36  ...

```

在训练过程中, 如果希望使用数据增强, 就将变量 `is_train` 设置为 `True`, 不需要则设置为 `False` 即可。

3.6.3 ResNet34+BERT

首先对原有的 ResNet 网络修改, 定义一个更深的 ResNet34 网络, 并且将文本编码器修改为预训练的 BERT 模型 (base)。修改代码如下:

```

1  import torch.nn as nn
2  import torch.nn.functional as F
3  from transformers import BertModel, BertTokenizer
4
5
6  class BERTTextEncoder(nn.Module):
7      def __init__(self, embed_dim=256, freeze_bert=True):
8          """
9          BERT文本编码器
10         参数:
11         - embed_dim: 输出特征维度
12         - max_length: 最大序列长度
13         - freeze_bert: 是否冻结BERT权重 (默认冻结)
14         """
15         super().__init__()

```

```

16 # 初始化BERT模型和tokenizer
17 self.bert = BertModel.from_pretrained('bert-base-uncased')
18 self.tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
19
20 # BERT输出维度 (768) 到目标维度的投影层
21 bert_hidden_size = self.bert.config.hidden_size
22 self.projection = nn.Linear(bert_hidden_size, embed_dim)
23
24 # 默认冻结BERT权重
25 if freeze_bert:
26     for name, param in self.bert.named_parameters():
27         if 'encoder.layer' in name:
28             layer_idx = int(name.split('.')[2]) # 层号在第2个拆分位置 (索引2)
29             if layer_idx < 9:
30                 param.requires_grad = False # 冻结前9层
31             else:
32                 param.requires_grad = True # 解冻最后3层
33         else:
34             # 非encoder.layer的参数 (如CLS token、嵌入层等) 默认解冻
35             param.requires_grad = False
36
37 def forward(self, input_ids, attention_mask=None):
38     outputs = self.bert(input_ids = input_ids, attention_mask = attention_mask)
39     pooled_output = outputs.pooler_output
40     return F.normalize(self.projection(pooled_output), p=2, dim=1)

```

Listing 1: bert_encoder.py

```

1 import torch.nn as nn
2 from .lstm_custom import LSTMTextEncoder
3 from .transformer_encoder import TransformerTextEncoder
4 from .bert_encoder import BERTTextEncoder
5 import os
6 os.environ["HF_ENDPOINT"] = "https://hf-mirror.com"
7
8
9 class TextEncoder(nn.Module):
10     def __init__(self, vocab_size, embed_dim=256, hidden_dim=512, encoder_type='transformer'):
11         """
12         参数说明:
13         - vocab_size: 词表大小, 对于BERT可忽略 (保留是为了接口兼容)
14         - embed_dim: 嵌入维度 (同时也是目标输出维度)
15         - hidden_dim: LSTM 隐藏层维度 (仅在 encoder_type='lstm' 时有效)
16         - encoder_type: 指定使用哪种编码器, 可选 'lstm' 或 'transformer' 或 'bert'
17         """
18         super().__init__()
19         self.encoder_type = encoder_type
20
21         if encoder_type == 'lstm':
22             # 使用手写版 LSTMTextEncoder, 注意此处 LSTMTextEncoder 内部已包含词嵌入层和全连接
23             # 映射
24             self.encoder = LSTMTextEncoder(vocab_size, embed_dim, hidden_dim)
25         elif encoder_type == 'transformer':
26             print("true")
27             # 使用手写版 TransformerTextEncoder, 注意内部参数可根据需求调整, 如头数、最大序列
28             # 长度等
29             self.encoder = TransformerTextEncoder(vocab_size, embed_dim, num_heads=4,
30 padding_idx=0, max_len=100)
31         elif encoder_type == 'bert':
32             # 使用手写版 BERTTextEncoder, 注意内部参数可根据需求调整
33             self.encoder = BERTTextEncoder(embed_dim, freeze_bert=True)
34             self.head = nn.Sequential(

```

```

32         nn.Linear(embed_dim, embed_dim),
33         nn.ReLU(),
34         nn.Dropout(0.1),
35         nn.LayerNorm(embed_dim)
36     )
37     else:
38         raise ValueError("Unsupported encoder_type: choose either 'lstm' , 'transformer'
39         or 'bert'")
40
41 def forward(self, captions):
42     # 直接调用内部 encoder 模块进行前向传播
43     if self.encoder_type == 'bert':
44         input_ids = captions
45         embeddings = self.encoder(input_ids)
46         embeddings = self.head(embeddings)
47         return embeddings
48     else:
49         # 其他编码器输入为单一 token ID 序列
50         return self.encoder(captions) # 输出形状: [batch, embed_dim]

```

Listing 2: text_encoder.py

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 # ----- 任务一：实现 BasicBlock 模块 -----
5
6 class BasicBlock(nn.Module):
7     def __init__(self, in_channels, out_channels, stride=1, downsample=None):
8         super().__init__()
9         # 实现第一个 3x3 卷积层, 包含 stride、padding=1, bias=False;
10         self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=3, stride=stride,
11                                 padding=1, bias=False)
12
13         # 【填空区域】：对 conv1 的输出接 BatchNorm
14         self.bn1 = nn.BatchNorm2d(out_channels)
15
16         # 定义 ReLU 激活, inplace 设为 True
17         self.relu = nn.ReLU(inplace=True)
18
19         # 【填空区域】：实现第二个 3x3 卷积层, 步长默认为1
20         self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3, stride=1, padding=1,
21                                 bias=False)
22
23         # conv2 的输出接 BatchNorm
24         self.bn2 = nn.BatchNorm2d(out_channels)
25
26         self.downsample = downsample
27         # 请完成 forward 函数
28         def forward(self, x):
29
30             identity = x
31
32             out = self.conv1(x)
33             out = self.bn1(out)
34             out = self.relu(out)
35
36             out = self.conv2(out)
37             out = self.bn2(out)
38
39             # 如果需要下采样, 对 identity 也进行转换
40             if self.downsample is not None:

```

```

39         identity = self.downsample(x)
40
41         out = out + identity # 残差连接
42         out = self.relu(out)
43
44         return out
45
46 class ResNet18(nn.Module):
47     def __init__(self, embed_dim=256):
48         super().__init__()
49         self.in_channels = 64
50         self.conv1 = nn.Conv2d(3, 64, kernel_size=7, stride=2, padding=3, bias=False)
51         self.bn1 = nn.BatchNorm2d(64)
52         self.relu = nn.ReLU(inplace=True)
53         self.maxpool = nn.MaxPool2d(kernel_size=3, stride=2, padding=1)
54         self.layer1 = self._make_layer(64, 2, stride=1)
55         self.layer2 = self._make_layer(128, 2, stride=2)
56         self.layer3 = self._make_layer(256, 2, stride=2)
57         self.layer4 = self._make_layer(512, 2, stride=2)
58         self.avgpool = nn.AdaptiveAvgPool2d((1, 1))
59         self.fc = nn.Linear(512, embed_dim)
60
61     def _make_layer(self, out_channels, blocks, stride):
62         downsample = None
63         if stride != 1 or self.in_channels != out_channels:
64             downsample = nn.Sequential(
65                 nn.Conv2d(self.in_channels, out_channels, kernel_size=1, stride=stride, bias=
False),
66                 nn.BatchNorm2d(out_channels)
67             )
68         layers = [BasicBlock(self.in_channels, out_channels, stride, downsample)]
69         self.in_channels = out_channels
70         for _ in range(1, blocks):
71             layers.append(BasicBlock(out_channels, out_channels))
72         return nn.Sequential(*layers)
73
74     def forward(self, x):
75         x = self.relu(self.bn1(self.conv1(x)))
76         x = self.maxpool(x)
77         x = self.layer1(x)
78         x = self.layer2(x)
79         x = self.layer3(x)
80         x = self.layer4(x)
81         x = self.avgpool(x)
82         x = torch.flatten(x, 1)
83         x = self.fc(x)
84         return F.normalize(x, p=2, dim=1)
85
86
87 class ResNet34(nn.Module):
88     def __init__(self, embed_dim=256):
89         super().__init__()
90         self.in_channels = 64
91         self.conv1 = nn.Conv2d(3, 64, kernel_size=7, stride=2, padding=3, bias=False)
92         self.bn1 = nn.BatchNorm2d(64)
93         self.relu = nn.ReLU(inplace=True)
94         self.maxpool = nn.MaxPool2d(kernel_size=3, stride=2, padding=1)
95         self.layer1 = self._make_layer(64, 3, stride=1)
96         self.layer2 = self._make_layer(128, 4, stride=2)
97         self.layer3 = self._make_layer(256, 6, stride=2)
98         self.layer4 = self._make_layer(512, 3, stride=2)
99         self.avgpool = nn.AdaptiveAvgPool2d((1, 1))

```

```

100     self.fc = nn.Linear(512, embed_dim)
101
102     def _make_layer(self, out_channels, blocks, stride):
103         downsample = None
104         if stride != 1 or self.in_channels != out_channels:
105             downsample = nn.Sequential(
106                 nn.Conv2d(self.in_channels, out_channels, kernel_size=1, stride=stride, bias=
False),
107                 nn.BatchNorm2d(out_channels)
108             )
109         layers = [BasicBlock(self.in_channels, out_channels, stride, downsample)]
110         self.in_channels = out_channels
111
112         for _ in range(1, blocks):
113             layers.append(BasicBlock(out_channels, out_channels))
114         return nn.Sequential(*layers)
115
116     def forward(self, x):
117         x = self.relu(self.bn1(self.conv1(x)))
118         x = self.maxpool(x)
119         x = self.layer1(x)
120         x = self.layer2(x)
121         x = self.layer3(x)
122         x = self.layer4(x)
123         x = self.avgpool(x)
124         x = torch.flatten(x, 1)
125         x = self.fc(x)
126         return F.normalize(x, p=2, dim=1)

```

Listing 3: resnet_custom.py

这个地方一开始做训练的时候是将 BERT 模型全部冻结，只训练了线性分类层，但是经过多个 epoch 后发现训练性能较差，loss 减小慢，recall 指标提升非常少。因此采取了现在这种方案，即将 BERT 模型的前 9 层做冻结，后 3 层解冻。

并且在网上查阅相关资料，发现 BERT 使用的词表和实验中使用的词表不相同，因此还需要对 data_loader.py 文件进行修改，使用 BERT 模型自带的词表：

```

1 ...
2
3     def __getitem__(self, idx):
4         filename, caption = self.pairs[idx]
5         image_path = os.path.join(self.root_dir, filename)
6         image = Image.open(image_path).convert("RGB")
7         #image = self.transform(image)
8
9         # 图像变换（训练集/验证集共用）
10        if self.transform is not None:
11            image = self.transform(image)
12
13        # 文本增强（仅训练集）
14        if self.is_train:
15            # 对文本进行增强（例如：每句话随机替换20%的词，删除10%的词）
16            augmented_caption = self.text_augmenter.augment(caption)
17            caption = augmented_caption # 替换为增强后的文本
18
19        caption_ids = self.tokenizer.encode(caption)
20
21        # 使用BERT Tokenizer编码，生成input_ids和attention_mask
22        encoded = self.tokenizer(
23            caption,
24            max_length=self.max_len,

```

```
25         padding="max_length", # 补全到max_len
26         truncation=True,       # 截断过长文本
27         return_attention_mask=True,
28         return_tensors="pt"     # 返回PyTorch张量
29     )
30     attention_mask = encoded["attention_mask"].squeeze(0) # [max_len]
31     caption_ids = encoded["input_ids"].squeeze(0)
32
33     return image, caption_ids, attention_mask
34
35 ...
```

Listing 4: data_loader.py

在训练中可以看到，词表大小从原本的 8318 变成了 30522。

训练 40 个周期后，在训练过程中会发现，模型收敛速度较慢。最终训练结束后模型的性能指标如下：

```
Epoch 40/40: 100% | 252/252 [02:35<00:00, 1.62batch/s, loss=0.908]
Epoch 39: Learning Rate = 0.000790
Extracting embeddings: 100% | 506/506 [00:28<00:00, 18.86it/s]

Text -> Image Retrieval:
Recall@1: 11.12%
Recall@5: 33.37%
Recall@10: 48.11%

Image -> Text Retrieval:
Recall@1: 11.97%
Recall@5: 34.64%
Recall@10: 48.53%
Epoch [40/40]: Train loss: 0.8690 | Val loss: 0.1624
验证集 文本->图像 Recall: @1 11.12% | @5 33.37% | @10 48.11%
验证集 图像->文本 Recall: @1 11.97% | @5 34.64% | @10 48.53%
> 验证集 Recall@1 提升至: 11.12%
> Best model updated at epoch 40
```

对比不使用预训练模型的方案提升并不明显，可能原因是这个模型的瓶颈出现在了图像编码器上，而不是文本编码器。对该训练完的模型进行测试，得到的结果如下：

```
Vocabulary size: 30522
加载最佳模型进行最终测试...
最佳模型加载成功。
从检查点加载了Tokenizer词表用于评估和可视化。
在测试集上进行最终评估...
Extracting embeddings: 100% | 1012/1012 [00:27<00:00, 36.16it/s]

测试集性能:
文本->图像 Recall@1: 9.84%
文本->图像 Recall@5: 32.25%
文本->图像 Recall@10: 46.17%

测试集性能:
图像->文本 Recall@1: 11.37%
图像->文本 Recall@5: 32.92%
图像->文本 Recall@10: 47.11%
```

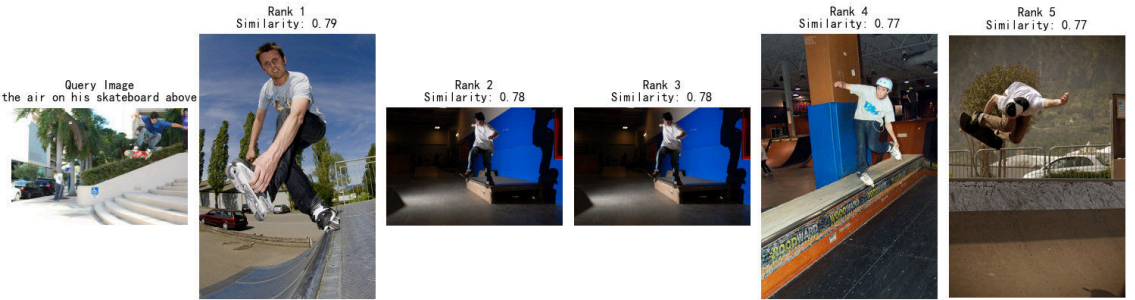


图 7: Top-5 相似图像结果


```

27         else:
28             param.requires_grad = True # 解冻最后2层
29         else:
30             # 非encoder.layer的参数（如CLS token、嵌入层等）默认解冻
31             param.requires_grad = False
32
33
34     self.projection = nn.Sequential(
35         nn.Linear(vit_output_dim, embed_dim),
36         nn.LayerNorm(embed_dim)
37     )
38
39     # 初始化投影层权重（可选）
40     nn.init.xavier_uniform_(self.projection[0].weight)
41     nn.init.constant_(self.projection[0].bias, 0)
42
43     def forward(self, images):
44         """
45         images: [batch, 3, 224, 224] 归一化后的图像张量
46         return: [batch, embed_dim] 归一化的图像嵌入
47         """
48         outputs = self.model(images)
49
50         # 提取[CLS] token并投影
51         cls_token = outputs.last_hidden_state[:, 0, :]
52         image_embed = self.projection(cls_token)
53         image_embed = nn.functional.normalize(image_embed, p=2, dim=1) # L2归一化
54
55     return image_embed

```

Listing 5: vit_custom.py

这里同样可以看到，我将 ViT 模型的后面 3 层解冻，前面 9 层冻结，原因和前一实验一致。

```

1 import torch.nn as nn
2 from .resnet_custom import ResNet18 # 替换 torch 的 resnet18
3 from .resnet_custom import ResNet34 # 替换 torch 的 resnet34
4 from .nfnet_custom import NFNet
5 from .vit_custom import VITEncoder
6
7 class ImageEncoder(nn.Module):
8     def __init__(self, embed_dim=256, encoder_type = 'resnet34'):
9         super().__init__()
10         self.encoder_type = encoder_type
11         if encoder_type == 'resnet18':
12             self.resnet = ResNet18() # 自己写的网络
13             self.fc = nn.Linear(256, embed_dim) # 将 ResNet 输出映射到共享空间
14
15         elif encoder_type == 'resnet34':
16             self.resnet = ResNet34() # 自己写的网络
17             self.fc = nn.Linear(256, embed_dim) # 将 ResNet 输出映射到共享空间
18
19         elif encoder_type == 'nfnet':
20             self.nfnet = NFNet() # 自己写的网络
21             self.fc = nn.Linear(256, embed_dim) # 将 NFNet 输出映射到共享空间
22
23         elif encoder_type == 'vit':
24             self.vit = VITEncoder(embed_dim)
25             self.fc = nn.Sequential(
26                 nn.Linear(embed_dim, embed_dim),
27                 nn.GELU(), # ViT 常用 GELU 激活

```

```

28         nn.Dropout(0.1),
29         nn.LayerNorm(embed_dim)
30     )
31
32     def forward(self, images):
33         if self.encoder_type == "resnet18":
34             features = self.resnet(images) # [batch, 256]
35             embeddings = self.fc(features) # [batch, embed_dim]
36         elif self.encoder_type == "resnet34":
37             features = self.resnet(images) # [batch, 256]
38             embeddings = self.fc(features) # [batch, embed_dim]
39         elif self.encoder_type == "nfnet":
40             features = self.nfnet(images) # [batch, 256]
41             embeddings = self.fc(features) # [batch, embed_dim]
42         elif self.encoder_type == "vit":
43             embeddings = self.vit(images) # [batch, embed_dim]
44             embeddings = self.fc(embeddings)
45         return embeddings

```

Listing 6: image_encoder.py

训练了 40 个周期后，模型的性能指标如下：

```

Epoch 3/20: 100% | 505/505 [03:34<00:00, 2.35batch/s, loss=0.0432]
Epoch 2: Learning Rate = 0.000400
Extracting embeddings: 100% | 1012/1012 [00:33<00:00, 30.05it/s]

Text -> Image Retrieval:
Recall@1: 35.06%
Recall@5: 65.29%
Recall@10: 75.70%

Image -> Text Retrieval:
Recall@1: 35.20%
Recall@5: 65.02%
Recall@10: 75.15%

Epoch [3/20]: Train Loss: 0.0561 | Val Loss: 0.0529
验证集 文本->图像 Recall: @1 35.06% | @5 65.29% | @10 75.70%
验证集 图像->文本 Recall: @1 35.20% | @5 65.02% | @10 75.15%
> 验证集 Recall@1 提升至: 35.06%
> Best model updated at epoch 3

```

可以看到，性能表现上还是非常不错的。进行测试，得到的测试性能指标如下：

```

PS C:\Users\skyhappy\Desktop\学校的故事\作业\媒体与认知上机作业> & c:/python313/python.exe c:/Users/skyhappy/Desktop/学校的故事/作业/媒体与认知上机作业/test.py
Vocabulary size: 8318
加载最佳模型进行最终测试...
Fast image processor class <class 'transformers.models.vit.image_processing_vit_fast.ViTImageProcessorFast'> is available for this model. Using slow image processor class. To use the fast image processor class set 'use_fast=True'.
true
最佳模型加载成功。
从检查点加载了Tokenizer词表用于评估和可视化。
在测试集上进行最终评估...
Extracting embeddings: 100% | 1012/1012 [00:33<00:00, 29.87it/s]

测试集性能:
文本->图像 Recall@1: 32.18%
文本->图像 Recall@5: 63.03%
文本->图像 Recall@10: 74.22%

测试集性能:
图像->文本 Recall@1: 32.43%
图像->文本 Recall@5: 62.65%
图像->文本 Recall@10: 74.10%

```

模型在测试集上的表现较好，和原训练性能相差不大，因此这种模型架构的复杂程度刚好适合该数据集。



图 9: Top-5 相似图像结果

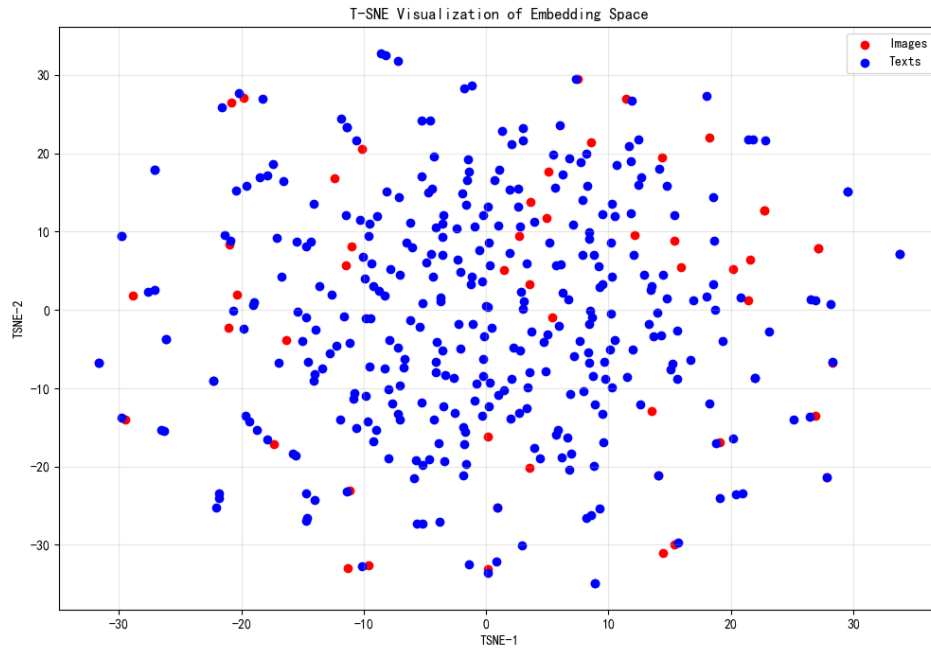


图 10: T-SNE 可视化结果

3.6.5 ViT+BERT

在做完以上尝试后,我将文本编码器和图像编码器全部换成了预训练模型,即 ViT+BERT 结构,并且对 train.py, data_loader.py 等程序做了相应的代码修改,具体代码已经放在了报告最后的源代码中。ViT 和 BERT 均解冻后 3 层。

在训练我发现,使用预训练模型的好处是性能提升非常明显,并且模型的收敛速度很快,一般训练到 10-20 个 epoch 后就会被早停,并且进行 15 轮数据增强训练同样也会被早停,因此未能画出模型训练的损失曲线。最终训练出来的模型性能指标为:



图 11: ViT+BERT 模型训练性能

虽然这张图并不是最佳性能的那一轮训练的指标,但是可以看到在早停前模型的性能相较于前两个实验,已经有了很大的提升。

在测试前,加载模型参数后先将模型的最佳性能打印出来:



图 12: ViT+BERT 模型最佳训练性能

运行 test.py 后，得到的结果如下:

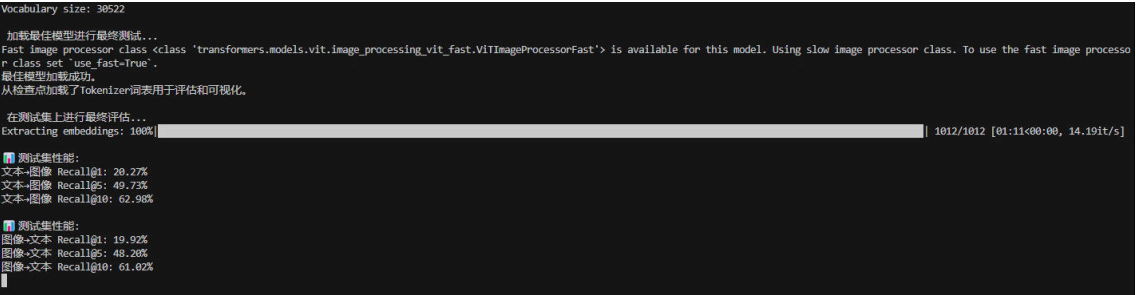


图 13: ViT+BERT 模型测试性能

可以发现文本、图像编码器均使用预训练模型这种方案在测试集上的性能有所降低，可能和模型结构太复杂导致对训练数据集过拟合了，总体性能表现差于只选择替换某一个编码器为预训练模型的方案，但是会优于不选择预训练模型方案的性能。



图 14: Top-5 相似图像结果

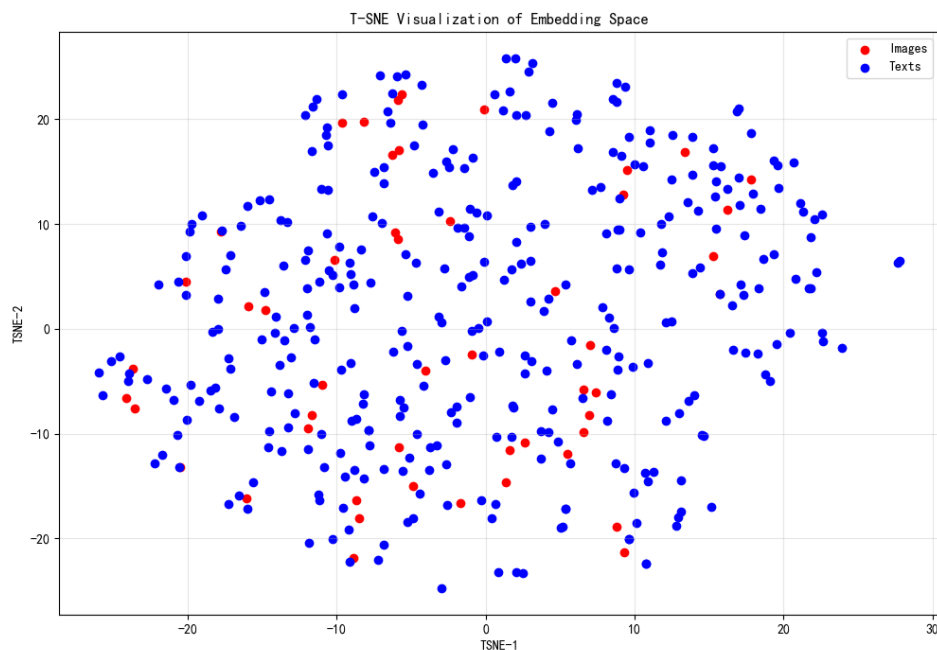


图 15: T-SNE 可视化结果

从 Top-5 相似图像结果可以看到，这个模型结构的表现还是很不错的，基本将相似的图片都找到了。

4 实验总结

中期验收前，主要将模型的基础模块代码完善并编写好了训练和测试代码，对 ResNet、LSTM、Transformer 的底层原理更加了解，同时自行编写 loss、train 和 test 代码也让我对训练模型的过程更加了解。

在做对模型的修改时，其实一开始出现了很多问题，比如使用预训练模型时冻结了所有层导致性能提升不明显，对 BERT 模型没有更换词表等导致训练收敛速度慢，并且性能反而有下降。在仔细查阅资料，检查代码后最终将这些问题一一解决。在做实验的过程中我也明白，训练模型并非是盲人摸象式的乱打乱撞，而是在仔细思考和度量后再进行大胆尝试。当然还有很多任务可以做，比如修改损失函数，更换更多的网络模型，研究预训练模型参数量大小对该数据集表现的变化等等。

最后感谢老师、助教在实验中的细心指导与帮助！很抱歉由于电脑、时间安排的原因未能在规定时间前提交完整作业，也感谢助教们的理解！

5 源代码

```

1 # train_and_eval.py
2
3 import os
4 os.environ["HF_ENDPOINT"] = "https://hf-mirror.com"
5 from torch.utils.data import DataLoader
6 from models.image_encoder import ImageEncoder
7 from models.text_encoder import TextEncoder
8 from loss import contrastive_loss
9 from data_loader import Flickr8kDataset
10 from utils import SimpleTokenizer
11 import torch
12 import torch.nn.functional as F
13 from tqdm import tqdm
14 import math
15 import numpy as np
16 import matplotlib.pyplot as plt
17 from PIL import Image
18 from sklearn.manifold import TSNE
19 import random
20 import nltk
21 #nltk.download('wordnet')
22 from transformers import BertTokenizer
23 import torchvision.transforms as T
24
25
26 #供同学们参考
27
28 plt.rcParams['font.sans-serif'] = ['SimHei']
29 plt.rcParams['axes.unicode_minus'] = False
30
31 def evaluate_top_k(img_encoder, txt_encoder, dataloader, device, topk=(1, 5, 10)):
32     img_encoder.eval()
33     txt_encoder.eval()
34
35     all_image_embeds = []
36     all_text_embeds = []
37
38     with torch.no_grad():
39         for images, captions_ids, attention_mask in tqdm(dataloader, desc="Extracting
40             embeddings"):
41             images = images.to(device)
42             captions_ids = captions_ids.to(device)
43             attention_mask = attention_mask.to(device)
44
45             image_embed = img_encoder(images) # [1, dim]
46             text_embed = txt_encoder(captions_ids, attention_mask) # [1, dim]
47
48             all_image_embeds.append(image_embed.cpu())
49             all_text_embeds.append(text_embed.cpu())
50
51     all_image_embeds = torch.cat(all_image_embeds, dim=0) # [N, D]
52     all_text_embeds = torch.cat(all_text_embeds, dim=0) # [N, D]
53
54     # 归一化
55     all_image_embeds = F.normalize(all_image_embeds, dim=1)
56     all_text_embeds = F.normalize(all_text_embeds, dim=1)
57
58     # 文本 -> 图像检索
59     sim_matrix = torch.matmul(all_text_embeds, all_image_embeds.T) # [N, N]

```

```

59     txt2img_ranks = torch.argsort(sim_matrix, dim=1, descending=True)
60
61     # 图像 -> 文本检索
62     sim_matrix_T = sim_matrix.T # [N, N]
63     img2txt_ranks = torch.argsort(sim_matrix_T, dim=1, descending=True)
64
65     def recall_at_k(ranks, topk):
66         recalls = []
67         for k in topk:
68             match = [i in ranks[i][:k] for i in range(len(ranks))]
69             recalls.append(np.mean(match))
70         return recalls
71
72     r_txt2img = recall_at_k(txt2img_ranks, topk)
73     r_img2txt = recall_at_k(img2txt_ranks, topk)
74
75     print("\n Text → Image Retrieval:")
76     for i, k in enumerate(topk):
77         print(f"Recall@{k}: {r_txt2img[i]*100:.2f}%")
78
79     print("\n Image → Text Retrieval:")
80     for i, k in enumerate(topk):
81         print(f"Recall@{k}: {r_img2txt[i]*100:.2f}%")
82
83
84     return r_txt2img, r_img2txt
85
86 def calc_val_loss(img_encoder, txt_encoder, dataloader, device):
87     img_encoder.eval() # 设置为评估模式
88     txt_encoder.eval() # 设置为评估模式
89     total_val_loss = 0 # 使用 total_val_loss 累加
90     with torch.no_grad(): # 在评估阶段不计算梯度
91         pbar = tqdm(dataloader, desc="计算验证损失", unit="batch", leave=False)
92         for images, captions_ids, attention_mask in pbar:
93             images = images.to(device)
94             captions_ids = captions_ids.to(device)
95             attention_mask = attention_mask.to(device)
96
97             image_embeds = img_encoder(images)
98             text_embeds = txt_encoder(captions_ids, attention_mask)
99             loss = contrastive_loss(image_embeds, text_embeds) # 调用对比损失函数
100             total_val_loss += loss.item()
101             pbar.set_postfix(loss=loss.item()) # 显示当前batch的损失
102     avg_val_loss = total_val_loss / len(dataloader)
103     return avg_val_loss
104
105
106 def adjust_learning_rate(optimizer, epoch, warmup_epochs = 10, base_lr = 1e-3):
107     """线性预热 + 余弦退火"""
108     if (epoch < warmup_epochs):
109         # 线性预热
110         lr = base_lr * (epoch + 1) / warmup_epochs
111     else:
112         # 余弦退火
113         progress = (epoch - warmup_epochs) / (100 - warmup_epochs)
114         lr = 0.5 * base_lr * (1 + math.cos(math.pi * progress))
115
116     for param_group in optimizer.param_groups:
117         param_group['lr'] = lr
118
119
120

```

```

121 def main():
122     # 设备设置
123     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
124     #print(torch.__version__)
125
126     # 文件路径, 根据实际调整
127     token_file = "C:/Users/skyhappy/Desktop/学校的破事/作业/媒认/媒体与认知上机作业/Flickr8k/
captions.txt" # 总的 captions 文件, 用于构建词表
128     train_token_file = "C:/Users/skyhappy/Desktop/学校的破事/作业/媒认/媒体与认知上机作业/
Flickr8k/train_captions.txt" # 训练集, 格式: image,caption
129     val_token_file = "C:/Users/skyhappy/Desktop/学校的破事/作业/媒认/媒体与认知上机作业/
Flickr8k/val_captions.txt" # 验证集
130
131     # 读取所有 caption 用于构建总词表 (假设以 tab 分隔, 如果不是, 请修改 split 参数)
132     with open(token_file, 'r', encoding="utf-8") as f:
133         lines = f.readlines()
134     captions = [line.strip().split(',')[1] for line in lines if line.strip()]
135
136     # 构建统一的 tokenizer
137     #tokenizer = SimpleTokenizer(captions, min_freq=1)
138     tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
139     vocab_size = len(tokenizer)
140     print(f"Vocabulary size: {vocab_size}")
141
142     # 训练集增强Transform (包含数据增强)
143     train_transform = T.Compose([
144         T.RandomRotation(degrees=15), # 随机旋转 (-15°~+15°)
145         T.RandomResizedCrop(size=(224, 224), scale=(0.8, 1.2)), # 随机缩放裁剪
146         T.ColorJitter(brightness=0.1, contrast=0.1, saturation=0.1, hue=0.1), # 色彩抖动
147         T.RandomHorizontalFlip(p=0.5), # 随机水平翻转 (50%概率)
148         T.ToTensor(), # 转为Tensor (0-255-0-1)
149         T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]) # 归一化 (预训练模
型常用)
150     ])
151
152     # 验证集/测试集基础Transform (仅预处理)
153     val_transform = T.Compose([
154         T.Resize((224, 224)), # 统一尺寸
155         T.ToTensor(),
156         T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]) # 与训练集保持一致
的归一化
157     ])
158
159     # 构建数据集与 DataLoader: 训练集、验证集、测试集
160     train_dataset = Flickr8kDataset(
161         root_dir="C:/Users/skyhappy/Desktop/学校的破事/作业/媒认/媒体与认知上机作业/Flickr8k/
images", # 图片所在目录
162         captions_file=train_token_file, # 训练集 captions 文件, 格式: image<TAB>caption
163         tokenizer=tokenizer,
164         transform=train_transform,
165         is_train=False
166     )
167     val_dataset = Flickr8kDataset(
168         root_dir="C:/Users/skyhappy/Desktop/学校的破事/作业/媒认/媒体与认知上机作业/Flickr8k/
images",
169         captions_file=val_token_file, # 验证集 captions 文件
170         tokenizer=tokenizer,
171         transform=val_transform,
172         is_train=False
173     )
174
175     train_dataloader = DataLoader(train_dataset, batch_size=128, shuffle=True, num_workers=4,

```

```

drop_last=True)
176 # 为保证评估稳定, 每个 batch 使用 batch_size=1
177 val_dataloader = DataLoader(val_dataset, batch_size=8, shuffle=False, num_workers=4,
drop_last=False)
178
179 # 构造模型 (设定 embed_dim=256)
180 embed_dim = 256
181 img_encoder = ImageEncoder(embed_dim = embed_dim, encoder_type = 'vit').to(device)
182 txt_encoder = TextEncoder(vocab_size, embed_dim = embed_dim, encoder_type = 'bert').to(
device)
183
184 # 优化器
185 bert_params = list(txt_encoder.encoder.bert.parameters())
186 other_params = (list(img_encoder.parameters()) +
187                 list(txt_encoder.encoder.projection.parameters()))
188
189 optimizer = torch.optim.AdamW(
190     [
191         {'params': bert_params, 'lr': 2e-5},
192         {'params': other_params, 'lr': 1e-4}
193     ]
194 )
195
196 best_val_loss = float('inf')
197 epochs = 40
198 real_epochs = 0
199 best_val_recall = 0
200 val_loss = 0
201 train_loss_history = []
202 val_loss_history = []
203 early_stop_patience = 15
204 early_stop_counter = 0
205
206 # 新增代码: 定义检查点路径和恢复训练标志
207 checkpoint_path = "C:/Users/skyhappy/Desktop/学校的破事/作业/媒认/媒体与认知上机作业/
best_metric_model.pth"
208 resume_training = False # 是否从检查点恢复训练
209
210 # 新增代码: 初始化训练相关变量 (会被检查点覆盖)
211 start_epoch = 0
212 best_val_recall = 0
213 best_val_loss = float('inf')
214 train_loss_history = []
215 val_loss_history = []
216
217 # 加载检查点
218 if os.path.exists(checkpoint_path):
219     print(f"发现检查点文件 {checkpoint_path}, 正在加载...")
220     checkpoint = torch.load(checkpoint_path, map_location=device, weights_only= False)
221
222     # 加载模型和优化器状态
223     img_encoder.load_state_dict(checkpoint['img_encoder_state_dict'])
224     txt_encoder.load_state_dict(checkpoint['txt_encoder_state_dict'])
225     optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
226
227     # 加载训练状态
228     start_epoch = checkpoint['epoch']
229     best_val_recall = checkpoint['best_val_recall_at_1']
230     best_val_loss = checkpoint['best_val_loss']
231     train_loss_history = checkpoint.get('train_loss_history', [])
232     val_loss_history = checkpoint.get('val_loss_history', [])
233

```

```

234     # 加载tokenizer词表（如果需要）
235     tokenizer.word2idx = checkpoint['tokenizer_vocab']
236
237     resume_training = True
238     print(f"成功加载检查点，将从 epoch {start_epoch} 继续训练")
239     print(f"当前最佳 Recall@1: {best_val_recall*100:.2f}% | 最佳验证损失: {best_val_loss:.4f}")
240
241     for epoch in range(epochs):
242         img_encoder.train()      # 设置为训练模式
243         txt_encoder.train()      # 设置为训练模式
244         epoch_loss = 0.0
245         pbar = tqdm(train_dataloader, desc=f"Epoch {epoch+1}/{epochs}", unit = "batch")
246         for images, captions_ids, attention_mask in pbar:
247             images = images.to(device)
248             captions_ids = captions_ids.to(device)
249             attention_mask = attention_mask.to(device)
250
251             image_embeds = img_encoder(images)      # [batch, embed_dim]
252             text_embeds = txt_encoder(captions_ids, attention_mask) # [batch, embed_dim]
253
254             loss = contrastive_loss(image_embeds, text_embeds)
255             optimizer.zero_grad()
256             loss.backward()
257             optimizer.step()
258
259
260             epoch_loss += loss.item()
261             pbar.set_postfix(loss = loss.item())
262
263         avg_train_loss = epoch_loss / len(train_dataloader)
264         train_loss_history.append(avg_train_loss) # 记录训练损失
265         val_loss = calc_val_loss(img_encoder, txt_encoder, val_dataloader, device)
266         val_loss_history.append(val_loss) # 记录验证损失
267
268
269         # 查看当前学习率
270         current_lr = optimizer.param_groups[0]['lr']
271         print(f"Epoch {epoch}: Learning Rate = {current_lr:.6f}")
272
273         # 手动更新学习率
274         adjust_learning_rate(optimizer, epoch)
275
276         r_txt2img, r_img2txt = evaluate_top_k(img_encoder, txt_encoder, val_dataloader, device, topk=(1, 5, 10))
277
278         print(f" Epoch [{epoch+1}/{epochs}]: Train Loss: {avg_train_loss:.4f} | Val Loss: {val_loss:.4f}")
279         print(f" 验证集 文本->图像 Recall: @1 {r_txt2img[0]*100:.2f}% | @5 {r_txt2img[1]*100:.2f}% | @10 {r_txt2img[2]*100:.2f}%")
280         print(f" 验证集 图像->文本 Recall: @1 {r_img2txt[0]*100:.2f}% | @5 {r_img2txt[1]*100:.2f}% | @10 {r_img2txt[2]*100:.2f}%")
281
282         # 如果验证集有改善，则保存最佳模型，这里选择loss以及recall参数
283         improved_metric = False
284         if r_txt2img[0] >= best_val_recall:
285             best_val_recall = r_txt2img[0]
286             print(f" > 验证集 Recall@1 提升至: {best_val_recall*100:.2f}%")
287             improved_metric = True
288
289         if val_loss <= best_val_loss:
290             best_val_loss = val_loss

```

```

291         print(f"    > 验证集损失降低至: {best_val_loss:.4f}")
292         improved_metric = True
293
294     if improved_metric:
295         checkpoint = {
296             'epoch': epoch + 1,
297             'img_encoder_state_dict': img_encoder.state_dict(),
298             'txt_encoder_state_dict': txt_encoder.state_dict(),
299             'optimizer_state_dict': optimizer.state_dict(),
300             'tokenizer_vocab': tokenizer.vocab, # 保存词表
301             'best_val_recall_at_1': best_val_recall,
302             'best_val_loss': best_val_loss,
303             'embed_dim': embed_dim, # 保存模型参数
304             'vocab_size': vocab_size # 保存模型参数
305         }
306         torch.save(checkpoint, checkpoint_path)
307         print(f"    > Best model updated at epoch {epoch+1} ")
308         early_stop_counter = 0 # 指标改善, 重置计数器
309     else:
310         early_stop_counter += 1 # 指标未改善, 计数器+1
311         print(f"    早停计数器: {early_stop_counter}/{early_stop_patience}")
312
313     # 触发早停条件
314     if early_stop_counter >= early_stop_patience:
315         print(f"    连续 {early_stop_patience} 轮无指标改善, 提前终止训练!")
316         real_epochs = epoch
317         break # 退出训练循环
318
319     print("\n训练完成。")
320
321     # 绘制loss曲线
322     plt.figure(figsize=(10, 6))
323     plt.plot(range(1, real_epochs + 1), train_loss_history, label='平均训练损失 (Avg Train Loss)')
324     plt.plot(range(1, real_epochs + 1), val_loss_history, label='平均验证损失 (Avg Val Loss)')
325     plt.xlabel('轮次 (Epochs)')
326     plt.ylabel('损失 (Loss)')
327     plt.title('训练和验证损失随轮次变化曲线')
328     plt.legend() # 显示图例
329     plt.grid(True) # 显示网格
330     plt.savefig("C:/Users/skyhappy/Desktop/学校的破事/作业/媒认/媒体与认知上机作业/loss_curve.png") # 将图像保存到文件
331     print("损失曲线已保存为 loss_curve.png")
332     plt.show()
333
334
335 if __name__ == "__main__":
336     main()

```

Listing 7: train.py

```

1 # test_and_visualization.py
2
3 import os
4 os.environ["HF_ENDPOINT"] = "https://hf-mirror.com"
5
6 from torch.utils.data import DataLoader
7 from models.image_encoder import ImageEncoder
8 from models.text_encoder import TextEncoder
9 from loss import contrastive_loss
10 from data_loader import Flickr8kDataset
11 from utils import SimpleTokenizer

```

```

12 import torch
13 import torch.nn.functional as F
14 from tqdm import tqdm
15 import numpy as np
16 import matplotlib.pyplot as plt
17 from PIL import Image
18 from sklearn.manifold import TSNE
19 import random
20 import torchvision.transforms as T
21 from transformers import BertTokenizer
22
23
24
25 #供同学们参考
26
27 plt.rcParams['font.sans-serif'] = ['SimHei']
28 plt.rcParams['axes.unicode_minus'] = False
29
30 def evaluate_top_k(img_encoder, txt_encoder, dataloader, device, topk=(1, 5, 10)):
31     img_encoder.eval()
32     txt_encoder.eval()
33
34     all_image_embeds = []
35     all_text_embeds = []
36
37     with torch.no_grad():
38         for images, captions_ids, attention_mask in tqdm(dataloader, desc="Extracting
39             embeddings"):
40             images = images.to(device)
41             captions_ids = captions_ids.to(device)
42             attention_mask = attention_mask.to(device)
43
44             image_embed = img_encoder(images) # [1, dim]
45             text_embed = txt_encoder(captions_ids, attention_mask) # [1, dim]
46
47             all_image_embeds.append(image_embed.cpu())
48             all_text_embeds.append(text_embed.cpu())
49
50     all_image_embeds = torch.cat(all_image_embeds, dim=0) # [N, D]
51     all_text_embeds = torch.cat(all_text_embeds, dim=0) # [N, D]
52
53     # 归一化
54     all_image_embeds = F.normalize(all_image_embeds, dim=1)
55     all_text_embeds = F.normalize(all_text_embeds, dim=1)
56
57     # 文本 -> 图像检索
58     sim_matrix = torch.matmul(all_text_embeds, all_image_embeds.T) # [N, N]
59     txt2img_ranks = torch.argsort(sim_matrix, dim=1, descending=True)
60
61     # 图像 -> 文本检索
62     sim_matrix_T = sim_matrix.T # [N, N]
63     img2txt_ranks = torch.argsort(sim_matrix_T, dim=1, descending=True)
64
65     def recall_at_k(ranks, topk):
66         recalls = []
67         for k in topk:
68             match = [i in ranks[i][:k] for i in range(len(ranks))]
69             recalls.append(np.mean(match))
70         return recalls
71
72     r_txt2img = recall_at_k(txt2img_ranks, topk)
73     r_img2txt = recall_at_k(img2txt_ranks, topk)

```

```

73
74
75     return r_txt2img, r_img2txt
76
77 def visualize_top_k(query_idx, img_encoder, txt_encoder, dataset, device, k=5):
78     img_encoder.eval() # 设置为评估模式
79     txt_encoder.eval() # 设置为评估模式
80
81     # 获取查询样本
82     query_image, query_caption_ids, query_attention_mask = dataset[query_idx]
83     query_caption = dataset.pairs[query_idx][1]
84
85     # 提取所有图像嵌入
86     all_image_embeds, all_img_paths = [], []
87     with torch.no_grad():
88         for img_path in dataset.img_paths:
89             image = Image.open(os.path.join(dataset.root_dir, img_path)).convert("RGB")
90             image = dataset.transform(image).unsqueeze(0).to(device)
91             embed = img_encoder(image).cpu()
92             all_image_embeds.append(embed)
93             all_img_paths.append(img_path)
94     all_image_embeds = torch.cat(all_image_embeds)
95
96     # 提取查询文本嵌入
97     with torch.no_grad():
98         caption_ids = query_caption_ids.unsqueeze(0).to(device)
99         attention_mask = query_attention_mask.unsqueeze(0).to(device)
100         text_embed = txt_encoder(caption_ids, attention_mask).cpu()
101
102     # 计算相似度
103     similarities = F.cosine_similarity(text_embed, all_image_embeds, dim=1)
104     topk_indices = similarities.argsort(descending=True)[:k]
105
106     # 可视化
107     plt.figure(figsize=(15, 5))
108     plt.subplot(1, k + 1, 1)
109     plt.imshow(Image.open(os.path.join(dataset.root_dir, dataset.img_paths[query_idx])))
110     plt.title("Query Image\n" + query_caption)
111     plt.axis('off')
112
113     for i, idx in enumerate(topk_indices):
114         plt.subplot(1, k+1, i+2)
115         img = Image.open(os.path.join(dataset.root_dir, all_img_paths[idx]))
116         plt.imshow(img)
117         plt.title(f"Rank {i+1}\nSimilarity: {similarities[idx]:.2f}")
118         plt.axis('off')
119     plt.tight_layout()
120     plt.savefig("C:/Users/skyhappy/Desktop/学校的破事/作业/媒认/媒体与认知上机作业/
121               retrieval_example.png")
122     plt.show()
123
124 def tsne_visualize(img_encoder, txt_encoder, dataloader, device, n_samples=200):
125     """TSNE嵌入空间可视化"""
126     img_encoder.eval() # 设置为评估模式
127     txt_encoder.eval() # 设置为评估模式
128
129     # 收集嵌入
130     image_embeds, text_embeds, labels = [], [], []
131     with torch.no_grad():
132         for idx, (images, captions_ids, attention_mask) in enumerate(dataloader):
133             if idx * dataloader.batch_size >= n_samples:
134                 break

```

```

134         images = images.to(device)
135         captions_ids = captions_ids.to(device)
136         attention_mask = attention_mask.to(device)
137
138         image_embeds.append(img_encoder(images).cpu())
139         text_embeds.append(txt_encoder(captions_ids, attention_mask).cpu())
140         labels.extend([f"Sample {idx*dataloader.batch_size+i}" for i in range(len(images))
141 ])
142
143 # 合并并降维
144 combined_embeds = torch.cat(image_embeds + text_embeds).numpy()
145 tsne = TSNE(n_components=2, random_state=42)
146 reduced = tsne.fit_transform(combined_embeds)
147
148 # 绘图
149 plt.figure(figsize=(12, 8))
150 split_point = len(image_embeds)
151 plt.scatter(reduced[:split_point, 0], reduced[:split_point, 1], c='r', label='Images')
152 plt.scatter(reduced[split_point:, 0], reduced[split_point:, 1], c='b', label='Texts')
153 plt.title("T-SNE Visualization of Embedding Space")
154 plt.xlabel("TSNE-1")
155 plt.ylabel("TSNE-2")
156 plt.legend()
157 plt.grid(alpha=0.3)
158 plt.savefig("C:/Users/skyhappy/Desktop/学校的破事/作业/媒认/媒体与认知上机作业/
159 tsne_visualization.png")
160 plt.show()
161
162 def calc_val_loss(img_encoder, txt_encoder, dataloader, device):
163     img_encoder.eval() # 设置为评估模式
164     txt_encoder.eval() # 设置为评估模式
165     total_val_loss = 0 # 使用 total_val_loss 累加
166     with torch.no_grad(): # 在评估阶段不计算梯度
167         pbar = tqdm(dataloader, desc="计算验证损失", unit="batch", leave=False)
168         for images, captions_ids, attention_mask in pbar:
169             images = images.to(device)
170             captions_ids = captions_ids.to(device)
171             attention_mask = attention_mask.to(device)
172
173             image_embeds = img_encoder(images)
174             text_embeds = txt_encoder(captions_ids, attention_mask)
175             loss = contrastive_loss(image_embeds, text_embeds) # 调用对比损失函数
176             total_val_loss += loss.item()
177             pbar.set_postfix(loss=loss.item()) # 显示当前batch的损失
178     avg_val_loss = total_val_loss / len(dataloader)
179     return avg_val_loss
180
181
182 def main():
183     # 设备设置
184     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
185
186     # 文件路径, 根据实际调整
187     token_file = "C:/Users/skyhappy/Desktop/学校的破事/作业/媒认/媒体与认知上机作业/Flickr8k/
188 captions.txt" # 总的 captions 文件, 用于构建词表
189     test_token_file = "C:/Users/skyhappy/Desktop/学校的破事/作业/媒认/媒体与认知上机作业/
190 Flickr8k/test_captions.txt" # 测试集
191
192     # 读取所有 caption 用于构建总词表 (假设以 tab 分隔, 如果不是, 请修改 split 参数)
193     with open(token_file, 'r', encoding="utf-8") as f:

```

```

192     lines = f.readlines()
193     captions = [line.strip().split(',')[1] for line in lines if line.strip()]
194
195
196     # 构建统一的 tokenizer
197     #tokenizer = SimpleTokenizer(captions, min_freq=1)
198     tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
199     vocab_size = len(tokenizer)
200     print(f"Vocabulary size: {vocab_size}")
201
202     # 验证集/测试集基础Transform (仅预处理)
203     test_transform = T.Compose([
204         T.Resize((224, 224)), # 统一尺寸
205         T.ToTensor(),
206         T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]) # 与训练集保持一致
207         的归一化
208     ])
209
210     test_dataset = Flickr8kDataset(
211         root_dir="C:/Users/skyhappy/Desktop/学校的破事/作业/媒认/媒体与认知上机作业/Flickr8k/
212         images",
213         captions_file=test_token_file, # 测试集 captions 文件
214         tokenizer=tokenizer,
215         transform=test_transform
216     )
217
218     test_dataloader = DataLoader(test_dataset, batch_size=4, shuffle=False, num_workers=4,
219     drop_last=False)
220
221     # 构造模型 (设定 embed_dim=256)
222     embed_dim = 256
223
224     # 训练完成, 最终在测试集上评估
225     print("\n 加载最佳模型进行最终测试...")
226     checkpoint = torch.load("C:/Users/skyhappy/Desktop/学校的破事/作业/媒认/媒体与认知上机作业
227     /best_metric_model.pth", map_location=device, weights_only = False) #加载最优模型
228
229     # 重新初始化模型结构以加载状态字典
230     loaded_embed_dim = checkpoint.get('embed_dim', embed_dim)
231     loaded_vocab_size = checkpoint.get('vocab_size', vocab_size)
232
233     final_img_encoder = ImageEncoder(embed_dim=loaded_embed_dim, encoder_type = 'vit').to(
234     device)
235     final_txt_encoder = TextEncoder(vocab_size=loaded_vocab_size, embed_dim=loaded_embed_dim,
236     encoder_type='bert').to(device)
237     final_img_encoder.load_state_dict(checkpoint['img_encoder_state_dict'])
238     final_txt_encoder.load_state_dict(checkpoint['txt_encoder_state_dict'])
239     print("最佳模型加载成功。")
240     tokenizer_for_eval_vis = SimpleTokenizer([], min_freq=1) # 创建空tokenizer
241     tokenizer_for_eval_vis.word2idx = checkpoint['tokenizer_vocab']
242
243     # 如果SimpleTokenizer依赖idx2word, 也需要重建
244     if hasattr(tokenizer_for_eval_vis, 'idx2word'):
245         tokenizer_for_eval_vis.idx2word = {v: k for k, v in tokenizer_for_eval_vis.
246         word2idx.items()}
247     print("从检查点加载了Tokenizer词表用于评估和可视化。")
248
249     # 使用加载的最佳模型进行评估
250     img_to_eval = final_img_encoder
251     txt_to_eval = final_txt_encoder
252     print("\n 在测试集上进行最终评估...")
253     r_txt2img_test, r_img2txt_test = evaluate_top_k(img_to_eval, txt_to_eval, test_dataloader,

```

```

        device)
247
248     print("\n 测试集性能:")
249     print(f"文本→图像 Recall@1: {r_txt2img_test[0]*100:.2f}%")
250     print(f"文本→图像 Recall@5: {r_txt2img_test[1]*100:.2f}%")
251     print(f"文本→图像 Recall@10: {r_txt2img_test[2]*100:.2f}%")
252
253     print("\n 测试集性能:")
254     print(f"图像→文本 Recall@1: {r_img2txt_test[0]*100:.2f}%")
255     print(f"图像→文本 Recall@5: {r_img2txt_test[1]*100:.2f}%")
256     print(f"图像→文本 Recall@10: {r_img2txt_test[2]*100:.2f}%")
257
258     # 可视化示例
259     random_idx = random.randint(0, len(test_dataset)-1)
260     visualize_top_k(random_idx, img_to_eval, txt_to_eval, test_dataset, device, k=5)
261
262     # T-SNE可视化
263     tsne_visualize(img_to_eval, txt_to_eval, test_dataloader, device)
264
265 if __name__ == "__main__":
266     main()

```

Listing 8: test.py

```

1  import os
2  import csv
3  import torch
4  from torch.utils.data import Dataset
5  from PIL import Image
6  import torchvision.transforms as T
7  from transformers import BertTokenizer
8  import nlpaug.augmenter.word as naw
9
10 class Flickr8kDataset(Dataset):
11     def __init__(self, root_dir, captions_file, tokenizer, transform=None, max_len=32,
12                 is_train=False):
13         self.root_dir = root_dir
14         self.transform = transform or T.Compose([
15             T.Resize((224, 224)),
16             T.ToTensor()
17         ])
18         self.tokenizer = tokenizer
19         self.max_len = max_len
20         self.is_train = is_train # 标记是否为训练集
21         self.pairs = self._load_pairs(captions_file)
22         self.img_paths = [fname for fname, _ in self.pairs]
23         self.captions = [cap for _, cap in self.pairs]
24
25         if self.is_train:
26             self.text_augmenter = TextAugmenter(replace_ratio=0.2, delete_ratio=0.1)
27
28     def _load_pairs(self, captions_file):
29         pairs = []
30         with open(captions_file, 'r', encoding='utf-8') as f:
31             reader = csv.reader(f)
32             for row in reader:
33                 if len(row) < 2:
34                     continue
35                 image_filename = row[0].strip()
36                 caption = row[1].strip()
37                 pairs.append((image_filename, caption))
38         return pairs

```

```

38
39     def __len__(self):
40         return len(self.pairs)
41
42     def __getitem__(self, idx):
43         filename, caption = self.pairs[idx]
44         image_path = os.path.join(self.root_dir, filename)
45         image = Image.open(image_path).convert("RGB")
46         #image = self.transform(image)
47
48         # 图像变换（训练集/验证集共用）
49         if self.transform is not None:
50             image = self.transform(image)
51
52         # 文本增强（仅训练集）
53         if self.is_train:
54             # 对文本进行增强（例如：每句话随机替换20%的词，删除10%的词）
55             augmented_caption = self.text_augmenter.augment(caption)
56             caption = augmented_caption # 替换为增强后的文本
57
58         caption_ids = self.tokenizer.encode(caption)
59
60         # 使用BERT Tokenizer编码，生成input_ids和attention_mask
61         encoded = self.tokenizer(
62             caption,
63             max_length=self.max_len,
64             padding="max_length", # 补全到max_len
65             truncation=True,      # 截断过长文本
66             return_attention_mask=True,
67             return_tensors="pt"   # 返回PyTorch张量
68         )
69         attention_mask = encoded["attention_mask"].squeeze(0) # [max_len]
70         caption_ids = encoded["input_ids"].squeeze(0)
71
72         return image, caption_ids, attention_mask
73
74
75
76
77 class TextAugmenter:
78     def __init__(self, aug_src='wordnet', replace_ratio=0.2, delete_ratio=0.1):
79         # 同义词替换增强器（基于 WordNet）
80         self.synonym_aug = naw.SynonymAug(aug_src=aug_src, aug_p=replace_ratio)
81         # 随机删除单词增强器
82         self.delete_aug = naw.RandomWordAug(aug_type='delete', aug_p=delete_ratio)
83
84     def augment(self, text):
85         # 随机选择一种增强方式（或叠加使用）
86         augmented = self.synonym_aug.augment(text) # 同义词替换
87         augmented = self.delete_aug.augment(augmented) # 随机删除
88         return augmented

```

Listing 9: data_loader.py

```

1 import torch
2 import torch.nn.functional as F
3 import tqdm
4 #请在这里写出对比损失函数
5 def contrastive_loss(image_embeds, text_embeds, temperature=0.07):
6     """
7     image_embeds, text_embeds: [batch, embed_dim]
8     """

```

```

9     device = image_embeds.device
10    batch_size = image_embeds.shape[0]
11
12    logits = torch.matmul(image_embeds, text_embeds.T) / temperature
13
14    # 创建正样本标签 [0,1,2,...,batch-1]
15    labels = torch.arange(batch_size, device=image_embeds.device)
16
17    # 计算图像->文本和文本->图像两个方向的交叉熵损失
18    loss_i2t = F.cross_entropy(logits, labels)
19    loss_t2i = F.cross_entropy(logits.T, labels) # 转置矩阵计算反向
20
21    # 最终损失取平均
22    loss = (loss_i2t + loss_t2i) / 2
23
24    return loss

```

Listing 10: loss.py

```

1  import torch
2  import torch.nn as nn
3  import torch.nn.functional as F
4  # ----- 任务一：实现 BasicBlock 模块 -----
5
6  class BasicBlock(nn.Module):
7      def __init__(self, in_channels, out_channels, stride=1, downsample=None):
8          super().__init__()
9          # 实现第一个 3x3 卷积层, 包含 stride、padding=1, bias=False;
10         self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=3, stride=stride,
11                                padding=1, bias=False)
12
13         # 【填空区域】：对 conv1 的输出接 BatchNorm
14         self.bn1 = nn.BatchNorm2d(out_channels)
15
16         # 定义 ReLU 激活, inplace 设为 True
17         self.relu = nn.ReLU(inplace=True)
18
19         # 【填空区域】：实现第二个 3x3 卷积层, 步长默认为1
20         self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3, stride=1, padding=1,
21                                bias=False)
22
23         # conv2 的输出接 BatchNorm
24         self.bn2 = nn.BatchNorm2d(out_channels)
25
26         self.downsample = downsample
27         # 请完成forward函数
28         def forward(self, x):
29
30             identity = x
31
32             out = self.conv1(x)
33             out = self.bn1(out)
34             out = self.relu(out)
35
36             out = self.conv2(out)
37             out = self.bn2(out)
38
39             # 如果需要下采样, 对 identity 也进行转换
40             if self.downsample is not None:
41                 identity = self.downsample(x)
42
43             out = out + identity # 残差连接

```

```

42         out = self.relu(out)
43
44         return out
45
46 class ResNet18(nn.Module):
47     def __init__(self, embed_dim=256):
48         super().__init__()
49         self.in_channels = 64
50         self.conv1 = nn.Conv2d(3, 64, kernel_size=7, stride=2, padding=3, bias=False)
51         self.bn1 = nn.BatchNorm2d(64)
52         self.relu = nn.ReLU(inplace=True)
53         self.maxpool = nn.MaxPool2d(kernel_size=3, stride=2, padding=1)
54         self.layer1 = self._make_layer(64, 2, stride=1)
55         self.layer2 = self._make_layer(128, 2, stride=2)
56         self.layer3 = self._make_layer(256, 2, stride=2)
57         self.layer4 = self._make_layer(512, 2, stride=2)
58         self.avgpool = nn.AdaptiveAvgPool2d((1, 1))
59         self.fc = nn.Linear(512, embed_dim)
60
61     def _make_layer(self, out_channels, blocks, stride):
62         downsample = None
63         if stride != 1 or self.in_channels != out_channels:
64             downsample = nn.Sequential(
65                 nn.Conv2d(self.in_channels, out_channels, kernel_size=1, stride=stride, bias=
False),
66                 nn.BatchNorm2d(out_channels)
67             )
68         layers = [BasicBlock(self.in_channels, out_channels, stride, downsample)]
69         self.in_channels = out_channels
70         for _ in range(1, blocks):
71             layers.append(BasicBlock(out_channels, out_channels))
72         return nn.Sequential(*layers)
73
74     def forward(self, x):
75         x = self.relu(self.bn1(self.conv1(x)))
76         x = self.maxpool(x)
77         x = self.layer1(x)
78         x = self.layer2(x)
79         x = self.layer3(x)
80         x = self.layer4(x)
81         x = self.avgpool(x)
82         x = torch.flatten(x, 1)
83         x = self.fc(x)
84         return F.normalize(x, p=2, dim=1)
85
86
87 class ResNet34(nn.Module):
88     def __init__(self, embed_dim=256):
89         super().__init__()
90         self.in_channels = 64
91         self.conv1 = nn.Conv2d(3, 64, kernel_size=7, stride=2, padding=3, bias=False)
92         self.bn1 = nn.BatchNorm2d(64)
93         self.relu = nn.ReLU(inplace=True)
94         self.maxpool = nn.MaxPool2d(kernel_size=3, stride=2, padding=1)
95         self.layer1 = self._make_layer(64, 3, stride=1)
96         self.layer2 = self._make_layer(128, 4, stride=2)
97         self.layer3 = self._make_layer(256, 6, stride=2)
98         self.layer4 = self._make_layer(512, 3, stride=2)
99         self.avgpool = nn.AdaptiveAvgPool2d((1, 1))
100        self.fc = nn.Linear(512, embed_dim)
101
102    def _make_layer(self, out_channels, blocks, stride):

```

```

103     downsample = None
104     if stride != 1 or self.in_channels != out_channels:
105         downsample = nn.Sequential(
106             nn.Conv2d(self.in_channels, out_channels, kernel_size=1, stride=stride, bias=
False),
107             nn.BatchNorm2d(out_channels)
108         )
109     layers = [BasicBlock(self.in_channels, out_channels, stride, downsample)]
110     self.in_channels = out_channels
111
112     for _ in range(1, blocks):
113         layers.append(BasicBlock(out_channels, out_channels))
114     return nn.Sequential(*layers)
115
116     def forward(self, x):
117         x = self.relu(self.bn1(self.conv1(x)))
118         x = self.maxpool(x)
119         x = self.layer1(x)
120         x = self.layer2(x)
121         x = self.layer3(x)
122         x = self.layer4(x)
123         x = self.avgpool(x)
124         x = torch.flatten(x, 1)
125         x = self.fc(x)
126     return F.normalize(x, p=2, dim=1)

```

Listing 11: resnet_custom.py

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 class LSTMTextEncoder(nn.Module):
5     def __init__(self, vocab_size, embed_dim=256, hidden_dim=256, num_layers=1, padding_idx=0)
:
6         super().__init__()
7
8         self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=padding_idx)
9         self.hidden_dim = hidden_dim
10
11
12         self.W_i = nn.Linear(embed_dim, hidden_dim)
13         # 初始化输入门参数, U_i: 上一个隐状态至隐状态
14         self.U_i = nn.Linear(hidden_dim, hidden_dim)
15
16         # 初始化遗忘门参数, W_f: 输入至隐状态
17         self.W_f = nn.Linear(embed_dim, hidden_dim)
18         # 初始化遗忘门参数, U_f: 上一个隐状态至隐状态
19         self.U_f = nn.Linear(hidden_dim, hidden_dim)
20
21         # 初始化输出门参数, W_o: 输入至隐状态
22         self.W_o = nn.Linear(embed_dim, hidden_dim)
23         # 初始化输出门参数, U_o: 上一个隐状态至隐状态
24         self.U_o = nn.Linear(hidden_dim, hidden_dim)
25
26         # 初始化候选状态参数, W_c: 输入至隐状态
27         self.W_c = nn.Linear(embed_dim, hidden_dim)
28         # 初始化候选状态参数, U_c: 上一个隐状态至隐状态
29         self.U_c = nn.Linear(hidden_dim, hidden_dim)
30
31         # 全连接层将最终的隐藏状态映射到 embed_dim (目标嵌入空间)
32         self.fc = nn.Linear(hidden_dim, embed_dim)
33     # 请完成 forward 函数

```

```

34 def forward(self, captions):
35     """
36     captions: [B, T], 表示批次 B 中每个句子的 token id 序列, T 为序列长度。
37     """
38     word_embedding = self.embedding(captions) # [B, T, E]
39     B, T, E = word_embedding.shape
40
41     h = torch.zeros(B, self.hidden_dim, device = word_embedding.device)
42     c = torch.zeros(B, self.hidden_dim, device = word_embedding.device)
43
44     for t in range(T):
45         x_t = word_embedding[:, t, :]
46
47         i_t = torch.sigmoid(self.W_i(x_t) + self.U_i(h)) # 输入门
48         f_t = torch.sigmoid(self.W_f(x_t) + self.U_f(h)) # 遗忘门
49         o_t = torch.sigmoid(self.W_o(x_t) + self.U_o(h)) # 输出门
50         c_hat_t = torch.tanh(self.W_c(x_t) + self.U_c(h))
51
52         c = f_t * c + i_t * c_hat_t # 更新细胞状态
53         h = o_t * torch.tanh(c) # 更新隐藏状态
54
55
56     # 将最终隐状态 h 通过全连接层映射至目标嵌入空间
57     out = self.fc(h) # [B, embed_dim]
58     # 对输出结果进行 L2 正则化归一化, 确保嵌入向量单位化
59     return F.normalize(out, p=2, dim=1)

```

Listing 12: lstm_custom.py

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 import math
5
6 class PositionalEncoding(nn.Module):
7     def __init__(self, d_model, max_len=100):
8         super().__init__()
9         pe = torch.zeros(max_len, d_model)
10        position = torch.arange(0, max_len).unsqueeze(1).float()
11        div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-math.log(10000.0) /
12        d_model))
13        pe[:, 0::2] = torch.sin(position * div_term)
14        pe[:, 1::2] = torch.cos(position * div_term)
15        self.pe = pe.unsqueeze(0)
16
17    def forward(self, x):
18
19        return x + self.pe[:, :x.size(1)].to(x.device)
20
21# ----- 填空题 1: 实现 MultiHeadSelfAttention 模块 -----
22class MultiHeadSelfAttention(nn.Module):
23    def __init__(self, embed_dim, num_heads):
24        super().__init__()
25        assert embed_dim % num_heads == 0, "embed_dim 必须能整除 num_heads"
26        self.num_heads = num_heads
27        self.head_dim = embed_dim // num_heads
28        self.qkv_proj = nn.Linear(embed_dim, embed_dim * 3)
29        self.out_proj = nn.Linear(embed_dim, embed_dim)
30
31# 请完成 forward 函数
32def forward(self, x, mask=None):
33    # x: [B, T, D], D=embed_dim
34    B, T, D = x.size()

```

```

33     qkv = self.qkv_proj(x) # [B, T, 3D]
34     q, k, v = torch.chunk(qkv, 3, dim=-1)
35
36     # 拆分
37     q = q.view(B, T, self.num_heads, self.head_dim).transpose(1, 2) # [B, nh, T, hd]
38     k = k.view(B, T, self.num_heads, self.head_dim).transpose(1, 2)
39     v = v.view(B, T, self.num_heads, self.head_dim).transpose(1, 2)
40
41     # 计算注意力分数
42     attn = (q @ k.transpose(-2, -1)) / math.sqrt(self.head_dim) # [B, nh, T, T]
43
44     if mask is not None:
45         attn = attn.masked_fill(mask.unsqueeze(1).unsqueeze(1) == 0, float('-inf'))
46     attn = F.softmax(attn, dim=-1)
47
48     # 计算注意力输出
49     attn_output = attn @ v # [B, nh, T, hd]
50     attn_output = attn_output.transpose(1, 2).contiguous().view(B, T, D) # [B, T, D]
51
52     # 通过输出投影层输出结果
53     return self.out_proj(attn_output)
54
55 class TransformerBlock(nn.Module):
56     def __init__(self, embed_dim, num_heads):
57         super().__init__()
58         self.attn = MultiHeadSelfAttention(embed_dim, num_heads)
59         self.ln1 = nn.LayerNorm(embed_dim)
60         # 初始化前馈网络, 先 Linear(embed_dim, embed_dim*4) -> ReLU -> Linear(embed_dim*4,
61         # embed_dim)
62         self.ff = nn.Sequential(
63             nn.Linear(embed_dim, embed_dim * 4),
64             nn.ReLU(),
65             nn.Linear(embed_dim * 4, embed_dim)
66         )
67         self.ln2 = nn.LayerNorm(embed_dim)
68
69     def forward(self, x, mask=None):
70         # 残差连接: 先 LayerNorm, 再 MultiHeadSelfAttention, 再加上输入
71         # 计算自注意力块输出, 并加上原输入, 实现残差连接
72         x = x + self.attn(self.ln1(x), mask)
73         # 同理, 对于前馈网络: 先 LayerNorm, 再前馈计算, 再加上输入
74         # 计算前馈网络输出, 并加上残差
75         x = x + self.ff(self.ln2(x))
76         return x
77
78 # ----- 填空题 2: 实现 TransformerTextEncoder 模块 -----
79 class TransformerTextEncoder(nn.Module):
80     def __init__(self, vocab_size, embed_dim=256, num_heads=4, num_layers=4, padding_idx=0,
81         max_len=100):
82         super().__init__()
83         # 初始化嵌入层
84         self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=padding_idx)
85         # 初始化位置编码器, 参数为 embed_dim 和 max_len
86         self.pos_encoder = PositionalEncoding(embed_dim, max_len=max_len)
87         self.layers = nn.ModuleList([TransformerBlock(embed_dim, num_heads) for _ in range(
88             num_layers)])
89         self.fc = nn.Linear(embed_dim, embed_dim)
90
91     # 请完成 forward 函数
92     def forward(self, captions):
93         # captions: [B, T]
94
95         # 输入嵌入和位置编码

```

```

92     x = self.embedding(captions) # [B, T, D]
93     x = self.pos_encoder(x)      # [B, T, D]
94
95     # 通过多层Transformer块
96     for layer in self.layers:
97         x = layer(x)              # [B, T, D]
98
99     # 全局平均池化
100    x = x.mean(dim=1)              # [B, D]
101    out = self.fc(x)
102    return F.normalize(out, p=2, dim=1)
103
104    # ----- 测试单元 -----
105    if __name__ == "__main__":
106        device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
107        vocab_size = 1000
108        # 随机生成输入: 形状为 [batch=4, T=20]
109        dummy_captions = torch.randint(0, vocab_size, (4, 20)).to(device)
110        # 实例化 TransformerTextEncoder, 要求使用 4 层 (num_layers=4)
111        model = TransformerTextEncoder(vocab_size, embed_dim=256, num_heads=4, num_layers=4,
112                                       padding_idx=0, max_len=100).to(device)
113        output = model(dummy_captions)
114        print("输出嵌入张量的形状: ", output.shape) # 期望输出: [4, 256]

```

Listing 13: transformer_encoder.py

```

1  import torch.nn as nn
2  import torch.nn.functional as F
3  from transformers import BertModel, BertTokenizer
4
5
6  class BERTTextEncoder(nn.Module):
7      def __init__(self, embed_dim=256, freeze_bert=True):
8          """
9          BERT文本编码器
10         参数:
11         - embed_dim: 输出特征维度
12         - max_length: 最大序列长度
13         - freeze_bert: 是否冻结BERT权重 (默认冻结)
14         """
15         super().__init__()
16         # 初始化BERT模型和tokenizer
17         self.bert = BertModel.from_pretrained('bert-base-uncased')
18         self.tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
19
20         # BERT输出维度 (768) 到目标维度的投影层
21         bert_hidden_size = self.bert.config.hidden_size
22         self.projection = nn.Linear(bert_hidden_size, embed_dim)
23
24         # 默认冻结BERT权重
25         if freeze_bert:
26             for name, param in self.bert.named_parameters():
27                 if 'encoder.layer' in name:
28                     layer_idx = int(name.split('.')[2]) # 层号在第2个拆分位置 (索引2)
29                     if layer_idx < 9:
30                         param.requires_grad = False # 冻结前10层
31                     else:
32                         param.requires_grad = True  # 解冻最后2层
33                 else:
34                     # 非encoder.layer的参数 (如CLS token、嵌入层等) 默认解冻
35                     param.requires_grad = False
36

```

```

37     def forward(self, input_ids, attention_mask=None):
38         outputs = self.bert(input_ids = input_ids, attention_mask = attention_mask)
39         pooled_output = outputs.pooler_output
40         return F.normalize(self.projection(pooled_output), p=2, dim=1)

```

Listing 14: bert_encoder.py

```

1  import torch.nn as nn
2  from .lstm_custom import LSTMTextEncoder
3  from .transformer_encoder import TransformerTextEncoder
4  from .bert_encoder import BERTTextEncoder
5  import os
6  os.environ["HF_ENDPOINT"] = "https://hf-mirror.com"
7
8
9  class TextEncoder(nn.Module):
10     def __init__(self, vocab_size, embed_dim=256, hidden_dim=512, encoder_type='transformer'):
11         """
12         参数说明:
13         - vocab_size: 词表大小,对于BERT可忽略(保留是为了接口兼容)
14         - embed_dim: 嵌入维度(同时也是目标输出维度)
15         - hidden_dim: LSTM 隐藏层维度(仅在 encoder_type='lstm' 时有效)
16         - encoder_type: 指定使用哪种编码器, 可选 'lstm' 或 'transformer' 或 'bert'
17         """
18         super().__init__()
19         self.encoder_type = encoder_type
20
21         if encoder_type == 'lstm':
22             # 使用手写版 LSTMTextEncoder, 注意此处 LSTMTextEncoder 内部已包含词嵌入层和全连接
23             # 映射
24             self.encoder = LSTMTextEncoder(vocab_size, embed_dim, hidden_dim)
25         elif encoder_type == 'transformer':
26             print("true")
27             # 使用手写版 TransformerTextEncoder, 注意内部参数可根据需求调整, 如头数、最大序列
28             # 长度等
29             self.encoder = TransformerTextEncoder(vocab_size, embed_dim, num_heads=4,
30 padding_idx=0, max_len=100)
31         elif encoder_type == 'bert':
32             # 使用手写版 BERTTextEncoder, 注意内部参数可根据需求调整
33             self.encoder = BERTTextEncoder(embed_dim, freeze_bert=True)
34             self.head = nn.Sequential(
35                 nn.Linear(embed_dim, embed_dim),
36                 nn.ReLU(),
37                 nn.Dropout(0.1),
38                 nn.LayerNorm(embed_dim)
39             )
40         else:
41             raise ValueError("Unsupported encoder_type: choose either 'lstm' , 'transformer'
42 or 'bert'")
43
44     def forward(self, captions, attention_mask):
45         # 直接调用内部 encoder 模块进行前向传播
46         if self.encoder_type == 'bert':
47             input_ids, attention_mask = captions, attention_mask
48             embeddings = self.encoder(input_ids, attention_mask)
49             embeddings = self.head(embeddings)
50             return embeddings
51         else:
52             # 其他编码器输入为单一 token ID 序列
53             return self.encoder(captions) # 输出形状: [batch, embed_dim]

```

Listing 15: text_encoder.py

```

1 import torch
2 import torch.nn as nn
3 from transformers import AutoImageProcessor
4 from transformers import ViTModel
5
6 import timm
7
8 class ViTEncoder(nn.Module):
9     def __init__(self, embed_dim=256):
10         super().__init__()
11         model_path = "C:/Users/skyhappy/Desktop/学校的破事/作业/煤认/vit-base-patch16-224-
            in21k"
12
13         # 加载预训练模型和处理器
14         self.processor = AutoImageProcessor.from_pretrained(model_path)
15         self.model = ViTModel.from_pretrained(model_path)
16
17         vit_output_dim = self.model.config.hidden_size
18
19
20
21         # 冻结所有预训练权重参数
22         for name, param in self.model.named_parameters():
23             if 'encoder.layer' in name:
24                 layer_idx = int(name.split('.')[2]) # 层号在第2个拆分位置 (索引2)
25                 if layer_idx < 9:
26                     param.requires_grad = False # 冻结前10层
27                 else:
28                     param.requires_grad = True # 解冻最后2层
29             else:
30                 # 非encoder.layer的参数 (如CLS token、嵌入层等) 默认解冻
31                 param.requires_grad = False
32
33
34         self.projection = nn.Sequential(
35             nn.Linear(vit_output_dim, embed_dim),
36             nn.LayerNorm(embed_dim)
37         )
38
39         # 初始化投影层权重 (可选)
40         nn.init.xavier_uniform_(self.projection[0].weight)
41         nn.init.constant_(self.projection[0].bias, 0)
42
43     def forward(self, images):
44         """
45         images: [batch, 3, 224, 224] 归一化后的图像张量
46         return: [batch, embed_dim] 归一化的图像嵌入
47         """
48         outputs = self.model(images)
49
50         # 提取[CLS] token并投影
51         cls_token = outputs.last_hidden_state[:, 0, :]
52         image_embed = self.projection(cls_token)
53         image_embed = nn.functional.normalize(image_embed, p=2, dim=1) # L2归一化
54
55         return image_embed

```

Listing 16: vit_custom.py

```

1 import torch.nn as nn
2 from .resnet_custom import ResNet18 # 替换 torch 的 resnet18

```

```

3 from .resnet_custom import ResNet34 # 替换 torch 的 resnet34
4 from .nfnet_custom import NFNet
5 from .vit_custom import VITEncoder
6
7 class ImageEncoder(nn.Module):
8     def __init__(self, embed_dim=256, encoder_type = 'resnet34'):
9         super().__init__()
10        self.encoder_type = encoder_type
11        if encoder_type == 'resnet18':
12            self.resnet = ResNet18() # 自己写的网络
13            self.fc = nn.Linear(256, embed_dim) # 将 ResNet 输出映射到共享空间
14
15        elif encoder_type == 'resnet34':
16            self.resnet = ResNet34() # 自己写的网络
17            self.fc = nn.Linear(256, embed_dim) # 将 ResNet 输出映射到共享空间
18
19        elif encoder_type == 'nfnet':
20            self.nfnet = NFNet() # 自己写的网络
21            self.fc = nn.Linear(256, embed_dim) # 将 NFNet 输出映射到共享空间
22
23        elif encoder_type == 'vit':
24            self.vit = VITEncoder(embed_dim)
25            self.fc = nn.Sequential(
26                nn.Linear(embed_dim, embed_dim),
27                nn.GELU(), # ViT 常用 GELU 激活
28                nn.Dropout(0.1),
29                nn.LayerNorm(embed_dim)
30            )
31
32    def forward(self, images):
33        if self.encoder_type == "resnet18":
34            features = self.resnet(images) # [batch, 256]
35            embeddings = self.fc(features) # [batch, embed_dim]
36        elif self.encoder_type == "resnet34":
37            features = self.resnet(images) # [batch, 256]
38            embeddings = self.fc(features) # [batch, embed_dim]
39        elif self.encoder_type == "nfnet":
40            features = self.nfnet(images) # [batch, 256]
41            embeddings = self.fc(features) # [batch, embed_dim]
42        elif self.encoder_type == "vit":
43            embeddings = self.vit(images) # [batch, embed_dim]
44            embeddings = self.fc(embeddings)
45        return embeddings

```

Listing 17: image_encoder.py